

회귀분석 — 한 권의 학습자료

부	다루는 것
1 부	회귀의 의미와 단순선형회귀
2 부	최소제곱법 — 손실함수와 정규방정식
3 부	단순선형회귀 — 다섯 가지 풀이법
4 부	다중회귀 — 행렬·표준화 계수·VIF· $R^2 = 1.0$ 의 의미
5 부	Cereals 자료의 PCA와 표준화의 필요성
6 부	PCR — 주성분 회귀
7 부	릿지 회귀 — L^2 규제와 닫힌 해
8 부	라소 회귀 — L^1 규제와 변수 선택
9 부	엘라스틱넷 — 두 규제의 결합
마무리	회귀가족 한눈에

1 부. 회귀의 의미와 단순선형회귀

1 장. 회귀란 무엇이며, 어디서 왔는가

회귀분석(regression analysis)의 핵심 질문은 단순하다 — **한 변수의 값을 알 때, 다른 변수의 값을 어떻게 예측·설명할 수 있는가**. 시리얼의 설탕 함량을 알면 소비자 등급이 어느 정도일지 짐작할 수 있다. 학생의 공부 시간을 알면 시험 점수가 어느 정도일지 짐작할 수 있다. 주택의 면적을 알면 가격이 어느 정도일지 짐작할 수 있다. 회귀는 이런 "한 변수로 다른 변수를 짐작하는 일"을 한 직선의 식으로 옮기는 도구다.

1.1 르장드르와 가우스 — 최소제곱법의 탄생 (1805-1809)

회귀의 수학적 뼈대 — 최소제곱법(method of least squares) — 은 1800년대 초의 두 수학자에게서 출발했다.

1805년, 프랑스의 르장드르(Adrien-Marie Legendre, 1752-1833)가 천체역학 책의 부록에서 혜성과 행성의 궤도 자료에서 "가장 잘 맞는 곡선"을 찾는 방법을 처음으로 발표했다. 관측 자료가 잡음을 가지고 있을 때, 곡선과 자료 점 사이의 거리의 **제곱의 합**을 최소화한다는 아이디어였다.

1809년, 독일의 가우스(Carl Friedrich Gauss, 1777-1855)가 "천체운동론"에서 같은 방법을 더 깊이 다듬었다. 가우스는 자신이 1795년부터 이 방법을 사용해 왔다고 주장했고, 더 중요하게는 — 측정 오차가 정규분포

(이른바 "가우스 분포") 를 따른다고 가정하면 최소제곱 추정값이 곧 최대우도 추정값과 일치한다는 사실을 증명했다. 최소제곱법이 단순한 "좋은 방법" 이 아니라 통계적으로 **최적의** 방법임이 이때 드러났다.

두 사람의 작업은 서로 거의 같은 시기에, 서로 모르게 이루어졌지만, 결과는 본질적으로 같았다. 오늘날 최소제곱법은 두 사람 모두의 이름으로 기억된다.

1.2 갈튼과 "회귀" 라는 용어 (1885)

"회귀(regression)" 라는 단어 자체는 르장드르도 가우스도 쓰지 않았다. 이 용어는 그로부터 약 80 년 뒤, 영국의 통계학자 골튼(Francis Galton, 1822-1911) 이 처음 사용했다.

1885 년, 갈튼은 부모와 자녀의 키 자료를 분석하면서 흥미로운 현상을 발견했다. 매우 큰 부모에게서 태어난 자녀는 평균적으로 부모보다 작았고, 매우 작은 부모에게서 태어난 자녀는 평균적으로 부모보다 컸다. 즉 자녀의 키가 "평균 쪽으로 되돌아가는(regression toward the mean)" 경향이 있었다. 갈튼은 이 현상을 묘사하기 위해 "평균으로의 회귀(regression to the mean)" 라는 표현을 썼고, 부모 키 $\rightarrow$ 자녀 키 의 관계를 직선으로 적합한 그 직선을 "회귀선(regression line)" 이라 불렀다.

본래 "회귀" 는 "되돌아간다" 라는 갈튼의 발견에 붙은 이름이었지만, 그 직선이 만들어지는 수학적 절차 — 두 변수 사이의 관계를 직선으로 모형화하는 절차 — 가 점차 일반화되면서 "회귀" 라는 단어가 그 절차 자체를 가리키는 표준 용어로 자리잡았다.

오늘날 회귀분석이라 부르면, 그것은 갈튼이 발견한 "평균으로 되돌아감" 의 현상이 아니라, **르장드르·가우스의 최소제곱법으로 두 변수 사이의 직선을 찾는 일** 을 가리킨다. 단어는 갈튼에게서 왔지만 알맹이는 르장드르·가우스에게서 왔다.

1.3 회귀가 답하는 세 가지 질문

회귀분석은 자료에 한 직선을 적합한 뒤, 그 직선으로 세 가지 질문에 답한다.

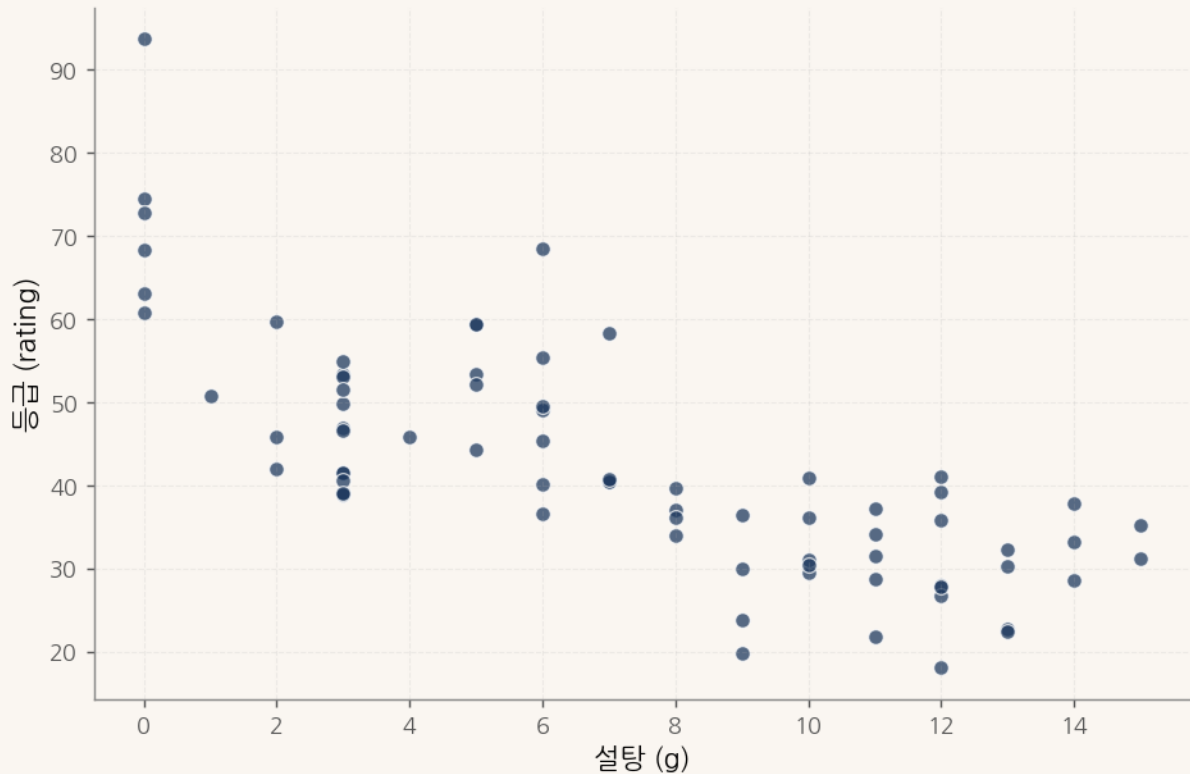
1. **예측** — x 의 값을 알 때, y 의 값은 얼마쯤 될까? 설탕이 8 g 인 새로운 시리얼이 있다면, 그 등급은 얼마로 짐작되는가?
2. **설명** — x 가 y 를 얼마나 "설명" 하는가? 설탕만 가지고 시리얼 등급의 변동을 몇 퍼센트나 잡아낼 수 있는가?
3. **관계의 방향과 강도** — x 가 1 단위 증가하면 y 는 평균적으로 얼마나 증가(또는 감소) 하는가? 설탕이 1 g 늘 때 등급이 몇 점 떨어지는가?

이 세 질문 모두에 대한 답이 적합한 직선의 "두 수" — 절편과 기울기 — 에 압축되어 있다.

2 장. 두 변수의 산점도

이제 본격적으로 자료를 본다. 시리얼 한 종은 영양 성분과 소비자 등급으로 묘사된다. 77 종의 시리얼 자료 중에서 두 변수 — **설탕 함량**(sugars, g) 과 **소비자 등급**(rating) — 만 골라 좌표평면에 찍어 본다.

시리얼 77 개 — 설탕 함량과 영양 등급



점들이 "왼쪽 위에서 오른쪽 아래로 비스듬히 내려오는 띠" 처럼 모인다. 설탕 함량이 많은 시리얼일수록 소비자 등급이 낮다는 경향이 한눈에 보인다. 설탕이 0 g 인 시리얼들은 등급이 60 점 이상, 설탕이 12 g 을 넘으면 등급이 30 점 아래로 떨어진다.

이 "비스듬히 내려가는 띠" 의 모양을 한 줄로 요약하고 싶다. 점들의 한가운데를 지나는 한 직선 — 그 직선의 식이 곧 자료의 한 줄짜리 요약이다.

3 장. 직선의 식과 두 모수

자료를 지나는 직선의 식을 다음과 같이 쓴다.

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

여기서 y_i 는 i 번째 시리얼의 실제 등급, x_i 는 같은 시리얼의 설탕 함량이다. β_0 는 직선이 y 축과 만나는 자리 — **절편**(intercept) 이고, β_1 은 직선의 기울기 — **회귀계수**(regression coefficient) 다. ε_i 는 직선이 점들과 정확히 일치하지 않을 때 생기는 **오차**(error) 항이다.

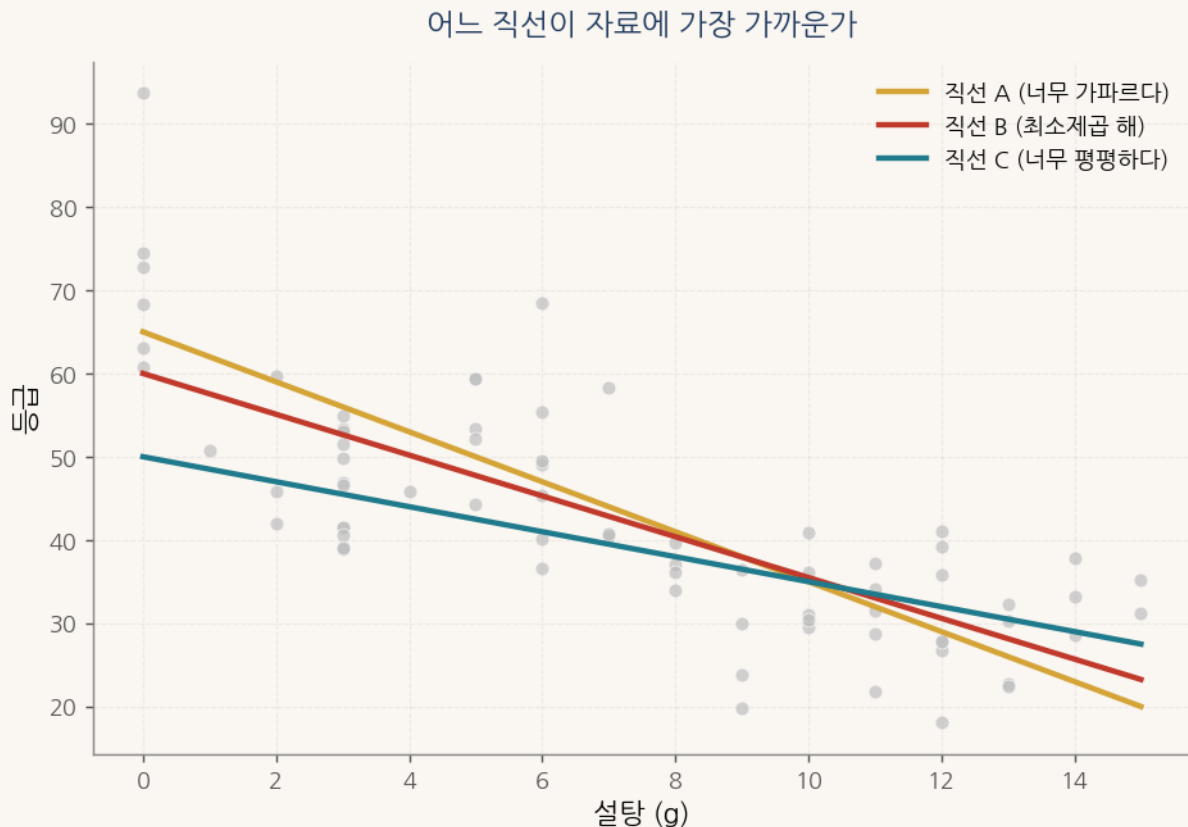
이 식이 가지는 의미를 한 줄씩 풀어 본다.

- β_0 는 $x = 0$ 일 때의 y 의 평균값이다. 설탕이 0 g 인 시리얼의 평균 등급에 해당한다.
- β_1 은 x 가 1 단위 늘 때 y 가 평균적으로 변하는 양이다. 설탕이 1 g 늘 때 등급이 평균적으로 몇 점 변하는지를 가리킨다.
- ε_i 는 모든 자료점이 한 직선 위에 정확히 놓이지 않는다는 사실을 인정하는 항이다. 영양 외의 다른 요인(시리얼 브랜드, 시판 시기, 광고, 측정 잡음) 등이 이 ε 에 모두 흡수된다.

직선 하나는 (β_0, β_1) 단 두 수로 완전히 결정된다. 자료에서 가장 잘 어울리는 한 직선을 찾는다는 것은 결국 가장 잘 어울리는 (β_0, β_1) 두 수를 찾는 일이다.

4 장. 어느 직선이 가장 좋은가 — 잔차의 정의

같은 산점도 위에 세 가지 직선을 그려 본다. 하나는 절편 60, 기울기 -1.5 인 직선 A. 하나는 (지금은 어떻게 구했는지 모르지만 가장 잘 맞는) 최소제곱 적합 직선. 하나는 절편 50, 기울기 -3.0 인 직선 B.



세 직선 모두 "왼쪽 위에서 오른쪽 아래로 내려가는" 추세는 잡고 있다. 그러나 자세히 보면 — A 는 점들의 위쪽으로 살짝 떠 있고, B 는 너무 가파르게 내려가고, 가운데 빨간 직선이 점들의 한가운데를 가장 잘 지나간다.

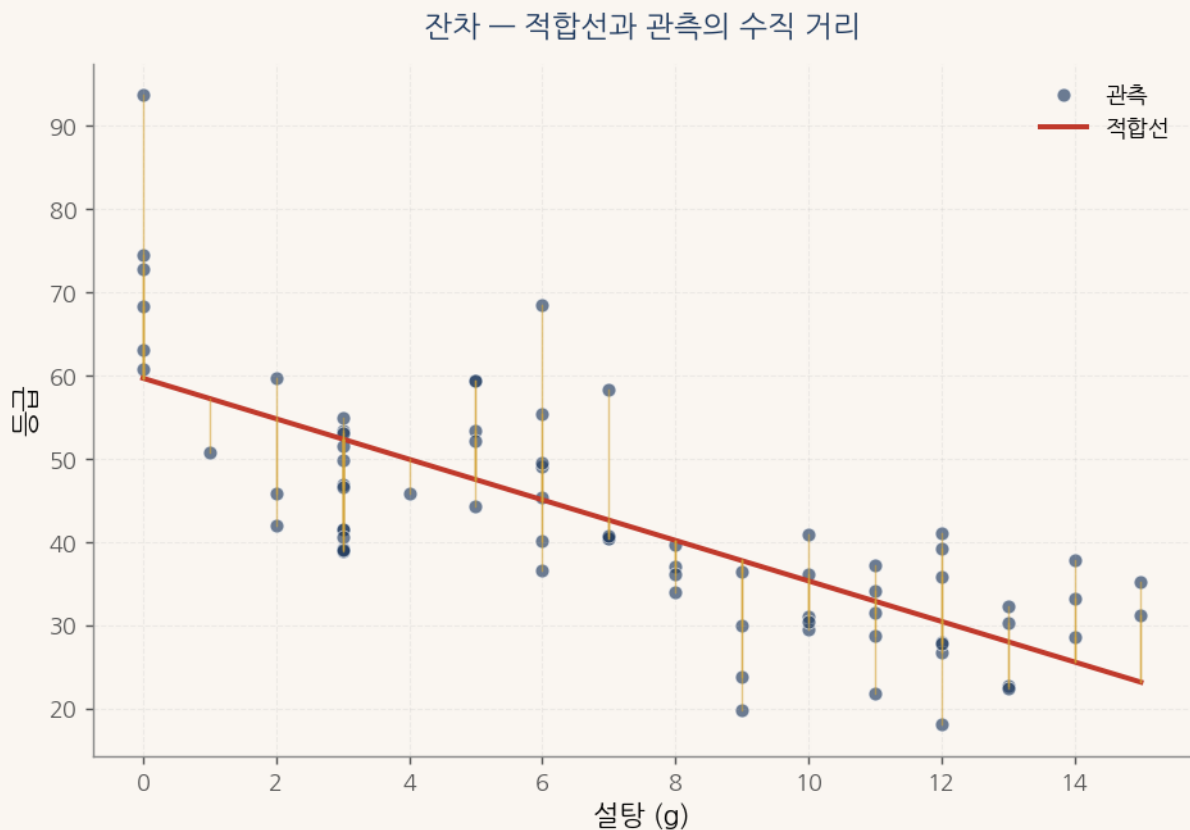
"가장 잘 어울리는" 이라는 직관을 수학적으로 옮기려면 "점과 직선 사이의 거리" 를 수로 표현해야 한다. 자료의 한 점 (x_i, y_i) 가 직선 위에서 가지는 "예상값" 은 다음과 같다.

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$$

이 예측값과 실제 y_i 의 차이가 "점과 직선 사이의 세로 거리"다. 이 차이를 **잔차**(residual)라 부른다.

$$r_i = y_i - \hat{y}_i = y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i)$$

잔차가 양수이면 점이 직선보다 위에 있다는 뜻이고, 음수이면 직선보다 아래에 있다는 뜻이며, 0이면 점이 직선 위에 정확히 놓였다는 뜻이다.



위 그림은 적합 직선과 시리얼 일곱 종의 잔차를 노란 세로선으로 보여준다. 100% Bran 시리얼은 적합 직선보다 약 34 점 더 높은 자리에 있고, Cap'n'Crunch는 직선보다 12.3 점 아래에 있다.

5 장. 잔차 제공 — 왜 절댓값이 아니라 제곱인가

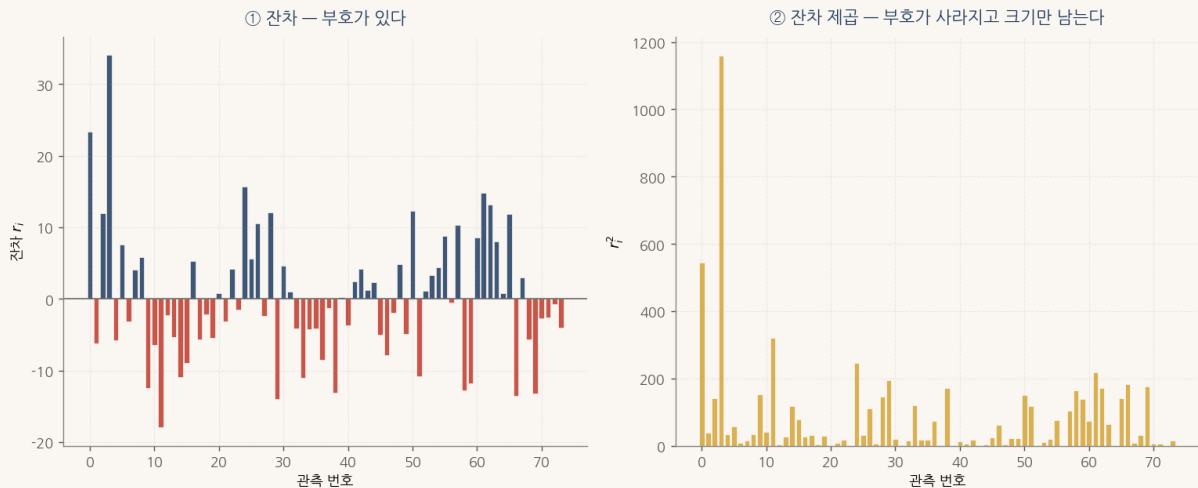
좋은 직선이란 "모든 점이 가까이 있는 직선" 이다. 즉 잔차의 절댓값이 전체적으로 작은 직선이다. 잔차를 그대로 더하면 양수와 음수가 상쇄돼 0 에 가까워지므로 좋은 척도가 못 된다. 두 가지 표준 대안이 있다.

1. 잔차의 절댓값을 더한다 — L^1 손실, 절댓값 회귀(absolute deviation regression).

2. 잔차를 제곱해서 더한다 — L^2 손실, 최소제곱법.

가우스가 1809 년에 선택한 방법이 후자다. 최소제곱법이 표준이 된 이유는 세 가지가 겹쳐서다. 첫째, 제곱은 모든 자리에서 미분 가능해서 수학적으로 다루기 쉽다. 절댓값은 0 에서 미분이 정의되지 않아 답을 닫힌 형태로 못 구한다. 둘째, 측정 오차가 정규분포를 따른다는 가정 아래에서 최소제곱이 최대우도 추정과 일치한다. 셋째, 답이 행렬 식 하나로 깔끔히 떨어진다.

따라서 "가장 좋은 직선" 은 잔차의 제곱의 합 — **잔차 제곱합**(Sum of Squared Errors, SSE) — 이 가장 작은 직선으로 정의된다.



왼쪽 패널은 점에서 직선까지의 세로 거리(잔차) 를 노란 세로선으로, 오른쪽 패널은 각 잔차의 제곱을 그 한 번으로 하는 사각형의 면적으로 보여준다. 사각형이 클수록 점이 직선에서 멀다는 뜻이고, 자료 전체의 "사각형 면적의 총합" 이 곧 SSE 다.

SSE 를 작게 만드는 (β_0, β_1) 한 쌍이 곧 최소제곱 적합 직선의 절편과 기울기다. 그 한 쌍을 어떻게 구하는지가 2 부의 주제다.

2 부. 최소제곱법 — 손실함수와 정규방정식

1 장. 손실함수 SSE 의 정의

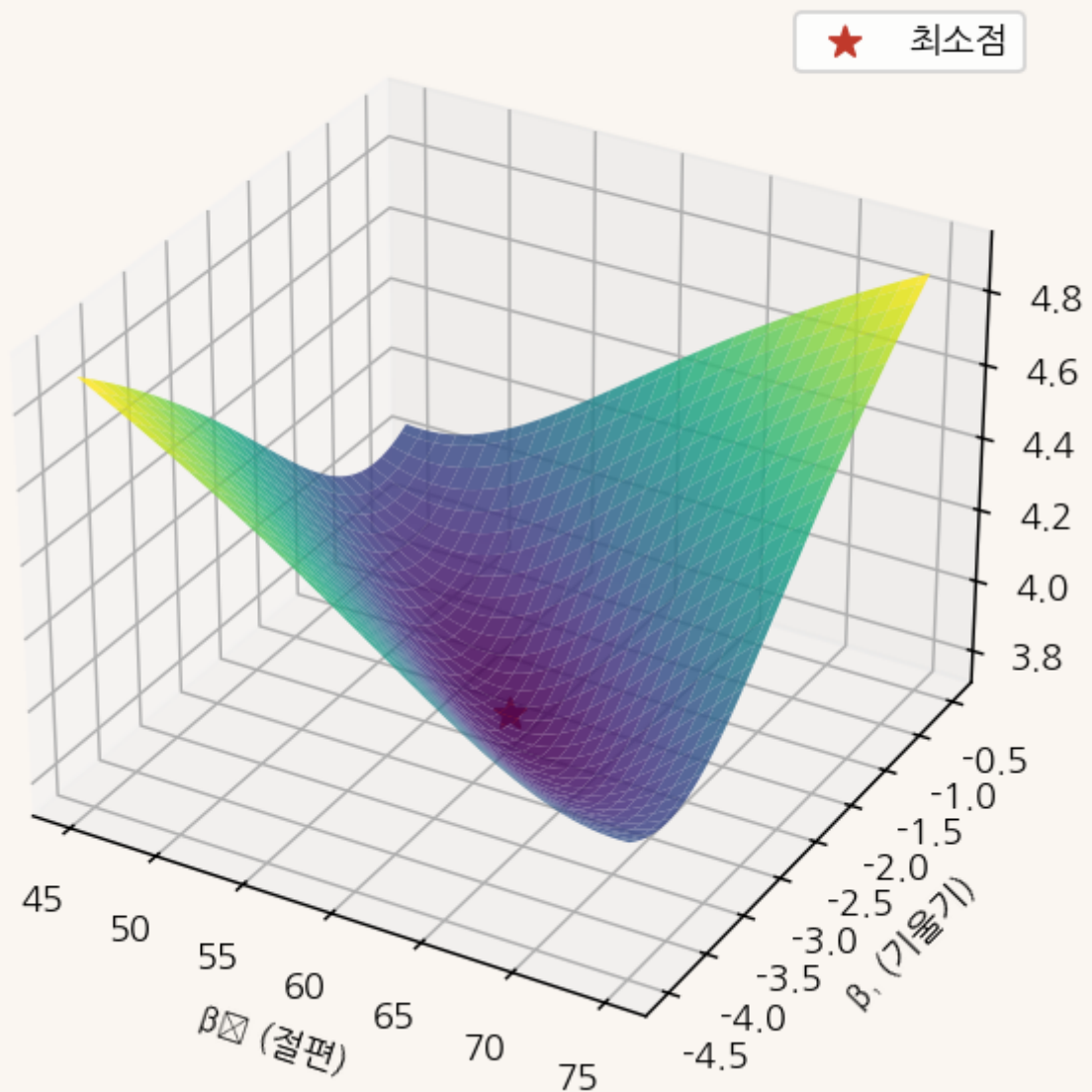
자료 n 개의 점에 대해, 직선 (β_0, β_1) 이 만드는 잔차 제곱의 합을 손실함수로 본다.

$$\text{SSE}(\beta_0, \beta_1) = \sum_{i=1}^n r_i^2 = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

SSE 는 (β_0, β_1) 의 함수다 — 직선을 어떻게 잡느냐에 따라 값이 달라진다. 좋은 직선을 찾는 일이란, SSE 를 가장 작게 만드는 (β_0, β_1) 한 쌍을 찾는 일과 같다.

(β_0, β_1) 의 평면 위에서 SSE 가 어떻게 변하는지를 그려 보면 다음과 같다.

SSE 손실 곡면 — 사발 모양의 한 점에서 최소



왼쪽이 3 차원 곡면, 오른쪽이 그 곡면을 위에서 본 등고선이다. 곡면의 모양이 "그릇" 처럼 한 점에서 가장 깊게 내려가고, 그 자리 외에서는 어디로 가든 위로 올라간다. 최소값을 갖는 단 하나의 자리가 곧 최적의 (β_0, β_1) 이다.

본 자료에서는 그 자리가 약 (59.85, -2.46) 부근이다.

2 장. 편미분 = 0 — 정규방정식의 유도

SSE 가 그릇 모양의 곡면이라면, 그릇의 가장 깊은 자리에서는 어느 방향으로 살짝 움직여도 SSE 값이 변하지 않는다 — 그 자리에서 곡면의 "기울기" 가 모든 방향으로 0 이라는 뜻이다. SSE 가 β_0 와 β_1 의 함수이므로, "모든 방향의 기울기가 0" 이라는 조건은 두 변수에 대한 **편미분**(partial derivative) 이 각각 0 이라는 식으로 옮겨진다.

먼저 β_0 에 대한 편미분.

$$\frac{\partial \text{SSE}}{\partial \beta_0} = -2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) = 0$$

다음 β_1 에 대한 편미분.

$$\frac{\partial \text{SSE}}{\partial \beta_1} = -2 \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i) = 0$$

각각 식의 양변을 2 로 나누고 합 기호 안의 항들을 분배하면 두 식이 깔끔하게 정리된다.

$$\sum y_i = n\beta_0 + \beta_1 \sum x_i, \quad \sum x_i y_i = \beta_0 \sum x_i + \beta_1 \sum x_i^2$$

이 두 식을 **정규방정식**(normal equations) 이라 부른다. 미지수 두 개 (β_0, β_1) 에 대한 일차연립방정식이다.

3 장. 정규방정식의 행렬 표기

정규방정식 두 줄을 한 줄로 묶으려면 행렬을 쓴다. 자료의 모든 점을 한 행렬에 담는다.

자료행렬 X 는 두 열로 구성된다. 첫 열은 모두 1, 둘째 열은 x_i 값들이다. 반응벡터 y 는 y_i 값들을 세로로 늘어놓은 것이다.

$$X = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix} \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

회귀계수도 벡터 $\beta = (\beta_0, \beta_1)^T$ 로 묶는다. 그러면 정규방정식 두 줄이 다음 한 줄로 떨어진다.

$$X^T X \hat{\beta} = X^T y$$

$X^T X$ 가 2×2 행렬, $X^T y$ 가 길이 2 인 벡터다. $X^T X$ 가 정칙(가역) 이라는 조건 하에서, 양변에 그 역행렬을 곱하면 한 줄짜리 닫힌 해가 나온다.

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

이 한 줄이 회귀분석의 모든 식의 출발점이다. 단순선형회귀에서나 다중회귀에서나 — 변수가 한 개든 백 개든 — 같은 식이 똑같이 적용된다. 차이는 단지 X 의 열 수가 늘어나고 β 의 길이가 길어진다는 것뿐이다.

4 장. 단순회귀의 깔끔한 공식

변수가 한 개인 경우(단순선형회귀) 에는, 정규방정식 두 줄을 손으로 풀어 β_0 와 β_1 의 닫힌 공식을 얻을 수 있다. 결과는 다음과 같다.

$$\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

위쪽 식이 기울기, 아래쪽 식이 절편이다. 기울기는 "x 와 y 의 공분산" 을 "x 의 분산" 으로 나눈 것에 정확히 비례한다 — 분자에는 두 변수가 함께 움직이는 정도가 들어가고, 분모에는 x 가 혼자 흩어진 정도가 들어간다. x 의 흩어짐에 비해 y 가 같이 흩어지는 정도가 곧 직선의 기울기다.

절편은 두 평균점 $(\bar{x}, \bar{y})$ 을 적합 직선이 항상 지나가도록 잡는다. $x = \bar{x}$ 일 때 $y = \bar{y}$ 가 되도록 β_0 가 결정된다는 뜻이다.

3 부. 단순선형회귀 — 다섯 가지 풀이법

같은 자료, 같은 식, 같은 답을 다섯 가지 다른 방법으로 풀어 본다. 각 방법이 "무엇을 어떻게 한다" 의 한 단계 한 단계를 끝까지 따라가며, 마지막에 다섯 답이 정확히 일치함을 확인한다.

사용하는 자료는 1 부에서 본 시리얼 자료다 — 설탕 함량 x (sugars) 에서 등급 y (rating) 을 예측하는 문제. 결측치를 제거한 $n = 76$ 개 시리얼.

1 장. 자료 준비

```
import pandas as pd
import numpy as np

# Cereals 자료를 GitHub raw URL 에서 불러온다. 이 한 코드 블록은
# Colab · Jupyter · 로컬 Python · 브라우저(Pyodide) 네 환경에서 모두 동작한다.
URL = ('https://raw.githubusercontent.com/leina99-lab/classes/main/'
      'AI%ED%94%84%EB%A1%9C%EA%B7%B8%EB%9E%98%EB%B0%8D/PCA/Cereals.csv')

try:
    from pyodide.http import open_url
    df = pd.read_csv(open_url(URL))
except ImportError:
    df = pd.read_csv(URL)

# sugars 와 rating 두 열만 사용하고, 결측치가 있는 행은 제거
df = df.dropna(subset=['sugars', 'rating'])

x = df['sugars'].values      # 설탕 함량 (g)
y = df['rating'].values      # 소비자 등급
n = len(x)

print(f'표본 수 n = {n}')
print(f'설탕 평균  $\bar{x}$  = {x.mean():.4f}')
print(f'등급 평균  $\bar{y}$  = {y.mean():.4f}')
```

```
표본 수 n = 76
설탕 평균  $\bar{x}$  = 7.0263
등급 평균  $\bar{y}$  = 42.5583
```

평균 설탕 7.03 g, 평균 등급 42.56 점이다. 이 두 평균점 ($\bar{x}, \bar{y}$) 은 적합 직선이 반드시 지나가는 자리다.

2 장. ① 공식법 — 손으로 한 줄씩

2 부 4 장의 공식 두 줄을 그대로 코드로 옮긴다.

```
xm, ym = x.mean(), y.mean()
Sxx = ((x - xm) ** 2).sum()      #  $\sum (x_i - \bar{x})^2$ 
Sxy = ((x - xm) * (y - ym)).sum() #  $\sum (x_i - \bar{x})(y_i - \bar{y})$ 

b1 = Sxy / Sxx                  # 기울기
b0 = ym - b1 * xm              # 절편

print(f' $\sum (x_i - \bar{x})^2 = \{Sxx:.4f\}$ ')
print(f' $\sum (x_i - \bar{x})(y_i - \bar{y}) = \{Sxy:.4f\}$ ')
print(f' $\hat{\beta}_1 = \{b1:.6f\}$ ')
print(f' $\hat{\beta}_0 = \{b0:.6f\}$ ')
```

```
 $\sum (x_i - \bar{x})^2 = 1437.9474$ 
 $\sum (x_i - \bar{x})(y_i - \bar{y}) = -3539.3927$ 
 $\hat{\beta}_1 = -2.461420$ 
 $\hat{\beta}_0 = 59.853017$ 
```

2.1 결과 해석

설탕 함량이 1 g 늘어날 때마다 시리얼 등급은 약 **2.46 점 떨어진다**. 설탕이 0 g 인 가상의 시리얼이 있다면 그 등급은 약 **59.85 점** 이 될 것으로 예측된다. 음의 기울기 -2.46 은 "설탕은 등급의 적" 이라는 자료의 메시지를 한 수로 압축한 것이다.

분자 $\sum(x_i - \bar{x})(y_i - \bar{y})$ 가 음수(-3539) 라는 사실이 곧 두 변수의 공분산이 음수라는 뜻이고, 그것이 "두 변수가 반대 방향으로 함께 움직인다" 라는 의미다. 분모 $\sum(x_i - \bar{x})^2 = 1437.95$ 가 x 의 흩어짐의 정도이고, 분자를 분모로 나누어 단위를 "등급 / 설탕g" 로 맞춘 것이 기울기다.

3 장. ② 행렬 곱셈 — $\beta = (X^T X)^{-1} X^T y$ 를 손으로

같은 자료에 2 부 3 장의 행렬 공식을 그대로 적용한다. 열의 처음을 1 로 둔 자료행렬 X 를 만들고, $X^T X$ 의 역행렬을 직접 계산한다.

```
# 1 열은 모두 1 (절편 항), 2 열은 x
X = np.column_stack([np.ones(n), x]) # 형태 (76, 2)

XtX = X.T @ X # (2, 2) 행렬
Xty = X.T @ y # (2,) 벡터

print('X^T X =')
print(XtX)
print('X^T y =', Xty)

XtX_inv = np.linalg.inv(XtX)
beta = XtX_inv @ Xty

print(f'\n β ^ = ({beta[0]:.6f}, {beta[1]:.6f})')
```

```
X^T X =
[[ 76.  534.]
 [ 534. 5190.]]
X^T y = [ 3234.4309 19186.7401]

β ^ = (59.853017, -2.461420)
```

3.1 결과 해석

$X^T X$ 의 (1,1) 원소가 76 — 표본 수 n — 이고, (1,2) 원소가 534 — $\sum x_i$ — 이고, (2,2) 원소가 5190 — $\sum x_i^2$ — 이다. 즉 $X^T X$ 는 "x 의 기본 통계량 네 가지" 를 한 행렬에 담은 것이다.

같은 식으로 $X^T y$ 의 첫 원소는 $\sum y_i$, 둘째 원소는 $\sum x_i y_i$ 다. 자료의 모든 정보가 이 작은 두 행렬·벡터에 압축되어 있고, 그 한 줄의 역행렬 곱셈으로 답이 떨어진다. 공식법과 정확히 같은 (59.853017, -2.461420) 이 나온다.

4 장. ③ Cramer 법칙 — 행렬식으로 푸는 일차연립방정식

$X^T X \beta = X^T y$ 는 미지수 두 개에 대한 일차연립방정식이다. 연립방정식을 푸는 고전적 방법이 Cramer 법칙이다. 계수행렬의 행렬식을 분모로, 각 열을 우변으로 교체한 행렬의 행렬식을 분자로 두면 각 미지수가 떨어진다.

```
# 계수행렬의 행렬식
det_XtX = XtX[0,0] * XtX[1,1] - XtX[0,1] * XtX[1,0]

#  $\hat{\beta}_0$  를 풀기 위한 행렬 —  $X^T X$  의 0 번 열을  $X^T y$  로 바꾼다
M0 = XtX.copy()
M0[:, 0] = Xty
det_M0 = M0[0,0] * M0[1,1] - M0[0,1] * M0[1,0]

#  $\hat{\beta}_1$  을 풀기 위한 행렬 —  $X^T X$  의 1 번 열을  $X^T y$  로 바꾼다
M1 = XtX.copy()
M1[:, 1] = Xty
det_M1 = M1[0,0] * M1[1,1] - M1[0,1] * M1[1,0]

b0 = det_M0 / det_XtX
b1 = det_M1 / det_XtX

print(f'det( $X^T X$ ) = {det_XtX:.4f}')
print(f' $\hat{\beta}_0 = \det(M_0)/\det(X^T X) = {b0:.6f}$ ')
print(f' $\hat{\beta}_1 = \det(M_1)/\det(X^T X) = {b1:.6f}$ ')

```

```
det( $X^T X$ ) = 109284.0000
 $\hat{\beta}_0 = \det(M_0)/\det(X^T X) = 59.853017$ 
 $\hat{\beta}_1 = \det(M_1)/\det(X^T X) = -2.461420$ 

```

4.1 결과 해석

$\det(X^T X) = 109284$ 이 0 이 아니라는 사실이 곧 $X^T X$ 가 가역이고 답이 유일하게 존재한다는 보장이다. 설탕 함량 x 가 모든 시리얼에서 같은 값이었다면 두 열이 선형종속이 되어 행렬식이 0 이 되고, 직선의 기울기를 정할 수 없어진다. 자료의 변동(분산) 이 회귀분석을 가능하게 하는 근본 조건임이 이 행렬식 한 수에 압축되어 있다.

Cramer 법칙은 2×2 와 3×3 까지는 손으로도 풀 만하지만, 변수가 많아지면 행렬식 자체의 계산이 매우 많아지기 때문에 실용성이 떨어진다. 답이 같다는 사실을 "보여주기" 위한 도구로서의 의미가 더 크다.

5 장. ④ NumPy 의 lstsq — 수치선형대수의 표준 도구

`np.linalg.lstsq` 는 "최소제곱 문제 $X\beta \approx y$ 를 풀어라" 라는 한 줄짜리 명령이다. 내부적으로는 QR 분해 또는 SVD 를 써서 $(X^T X)^{-1}$ 을 명시적으로 계산하지 않고도 답을 구한다 — 수치적으로 더 안정적이다.

```
beta, residuals_sse, rank, sv = np.linalg.lstsq(X, y, rcond=None)

print(f' $\hat{\beta} = ({\beta_0:.6f}, {\beta_1:.6f})$ ')
print(f'잔차 제곱합 SSE = {residuals_sse[0]:.4f}')
print(f' $X$  의 랭크 = {rank}')
print(f' $X$  의 특이값 = {sv}')

```

```
 $\hat{\beta} = (59.853017, -2.461420)$ 
잔차 제곱합 SSE = 6217.3615
 $X$  의 랭크 = 2
 $X$  의 특이값 = [72.79  6.39]

```

5.1 결과 해석

`lstsq` 가 한꺼번에 네 가지를 돌려준다 — 회귀계수, 잔차 제곱합 SSE, X 의 랭크, X 의 특이값. 랭크가 2라는 것은 "두 열(상수열, 설탕열)이 서로 독립이다"라는 뜻이고, 그것이 곧 유일한 최소제곱해가 존재한다는 보장이다.

특이값 72.79와 6.39 — 두 값의 비율(약 11배)이 자료의 "조건수"와 관련이 있다. 비율이 클수록 $X^T X$ 의 역행렬이 수치적으로 불안정해진다. 본 자료에서는 안정적이지만, 다중회귀로 가면 이 비율이 매우 커져 "다중공선성" 문제가 발생한다 — 5부에서 다시 본다.

6 장. ⑤ scikit-learn의 LinearRegression — 고수준 ML 인터페이스

scikit-learn은 머신러닝 라이브러리지만, 단순선형회귀도 같은 패턴 `fit` → `predict`로 제공한다.

```
from sklearn.linear_model import LinearRegression

# sklearn은 X를 2차원 배열로 받는다 — (n, 1)
reg = LinearRegression()
reg.fit(x.reshape(-1, 1), y)

print(f'β̂ (intercept_) = {reg.intercept_:.6f}')
print(f'β̂₁ (coef_[0]) = {reg.coef_[0]:.6f}')

# 예측
y_pred = reg.predict(x.reshape(-1, 1))
print(f'\n첫 시리얼 예측값: {y_pred[0]:.4f}, 실제값: {y[0]:.4f}')

# 적합도 R²
r2 = reg.score(x.reshape(-1, 1), y)
print(f'R² = {r2:.4f}')
```

```
β̂ (intercept_) = 59.853017
β̂₁ (coef_[0]) = -2.461420

첫 시리얼 예측값: 45.0844, 실제값: 68.4030
R² = 0.5835
```

6.1 결과 해석

sklearn도 같은 답을 준다. 내부적으로 NumPy의 `lstsq`와 비슷한 수치 선형대수를 사용하지만, "학습된 모형 객체"라는 개념을 제공한다는 차이가 있다. 같은 모형을 새 자료에 `predict`로 즉시 적용할 수 있고, 회귀와 분류·정규화·교차검증 같은 다른 도구들과 같은 인터페이스로 묶일 수 있다.

$R^2 = 0.5835$ 는 본 모형이 설탕 함량 한 변수만으로 시리얼 등급의 변동을 약 **58.4% 설명한다**는 뜻이다. 나머지 41.6%의 변동은 단백질, 식이섬유, 나트륨, 칼륨, 비타민 같은 다른 영양 변수와 측정 잡음에서 온다. 이 점이 4부에서 다중회귀로 넘어가야 할 동기를 만든다.

7 장. 다섯 답의 비교 — 정확히 같다

같은 자료에 다섯 가지 방법을 적용한 결과를 한 표에 모은다.

방법	β_0 (절편)	β_1 (기울기)
① 공식법	59.853017	-2.461420
② 행렬 곱셈 $(X^T X)^{-1} X^T y$	59.853017	-2.461420
③ Cramer 법칙	59.853017	-2.461420
④ NumPy <code>lstsq</code>	59.853017	-2.461420
⑤ scikit-learn <code>LinearRegression</code>	59.853017	-2.461420

소수점 여섯째 자리까지 정확히 같다. 같은 최소제곱 문제의 답을 다섯 가지 다른 길로 구했지만, 모두 "SSE가 최소가 되는 단 하나의 자리"에 도달했다는 뜻이다.

7.1 방법 선택의 기준 — 어느 것을 언제 쓰는가

상황	권장 방법
변수가 1 개, 손으로 풀이 과정을 보여줄 때	① 공식법
변수가 2~5 개, 행렬 곱셈을 직접 보여줄 때	② 행렬 곱셈
행렬식의 의미를 강조할 때 (교육용)	③ Cramer 법칙
변수가 많거나 $X^T X$ 가 거의 비가역일 때	④ NumPy <code>lstsq</code>
다른 ML 모형과 같은 인터페이스로 묶을 때	⑤ scikit-learn

실용에서는 ④와 ⑤가 표준이다. ① ~ ③은 "왜 답이 그런 모양으로 나오는가"를 설명하는 도구로서의 가치가 크다. 답이 같음을 한 번 확인하고 나면, 그 다음부터는 어느 도구를 쓰든 같은 결과가 나온다는 사실을 잊지 않는 것이 중요하다.

7.2 적합 직선의 한 줄 요약

다섯 풀이법이 모두 다음 한 줄을 가리킨다.

$$\text{rating} \approx 59.85 - 2.46 \times \text{sugars} \quad (R^2 = 0.58, n = 76)$$

설탕이 1 g 늘 때마다 등급이 2.46 점 떨어지고, 모형이 자료의 약 58%를 설명한다는 한 줄 요약이다. 나머지 42%를 잡으려면 변수를 추가해야 한다 — 그 자리가 4부 다중회귀다.

4 부. 다중회귀 — 행렬로 한 줄에

1 장. 왜 변수를 늘리는가 — 한 변수의 한계

3 부에서 설탕(sugars) 한 변수만으로 시리얼 등급을 설명했다. 결과는 $R^2 = 0.58$ — 자료의 약 58% 를 잡았다는 뜻이지만, 뒤집어 말하면 **42%의 변동을 설명하지 못했다**. 적합 직선에서 30 점 이상 벗어나는 시리얼이 여럿이고, 같은 설탕 함량(예: 3 g) 을 가진 시리얼들의 등급도 30 점 ~ 65 점 사이로 크게 다르다.

설탕 외에 또 무엇이 등급에 영향을 줄까? 자료에는 13 개의 영양 변수가 더 있다 — calories, protein, fat, sodium, fiber, carbo, potass, vitamins, weight, cups, shelf 등. 이들 중 어느 것이 설탕과 별도로 등급에 추가 정보를 주는지 모른다. 설탕 한 변수만 가지고는 "설탕이 같아도 식이섬유가 많은 시리얼은 등급이 높다" 같은 사실을 잡아낼 수 없다.

다중회귀(multiple regression) 는 여러 변수를 한꺼번에 고려해서, 각 변수의 "독립적인 기여도" 를 분리해 낸다. 설탕이 1 g 늘 때 등급이 떨어지는 정도를 — **다른 영양 성분이 모두 같다는 가정 아래** — 따로 잰다. 이 "다른 변수를 고정한 상태에서의 효과" 라는 개념이 다중회귀의 핵심이다.

2 장. 변수 두 개 — 평면이 된다

먼저 변수를 한 개 늘려서 두 변수의 경우부터 본다. 설탕과 식이섬유(fiber) 두 변수로 등급을 설명하는 모형은 다음과 같다.

$$y_i = \beta_0 + \beta_1 \cdot \text{sugars}_i + \beta_2 \cdot \text{fiber}_i + \varepsilon_i$$

기하적으로 보면 — 이 식은 (x_1, x_2, y) 의 3 차원 공간에서 한 **평면**의 식이다. 단순회귀가 (x, y) 평면에서의 한 직선이었다면, 변수가 둘이면 그 "직선" 이 한 차원 올라간 "평면" 이 된다. 자료의 점들이 3 차원 공간에 흩어져 있고, 그 점들 한가운데를 지나는 평면 한 장의 식을 찾는 일이다.

평면의 식은 세 수 $(\beta_0, \beta_1, \beta_2)$ 로 결정된다. β_1 은 "설탕이 1 g 늘 때 등급이 변하는 양 — 단, 식이섬유가 같다는 조건 아래" 이고, β_2 는 "식이섬유가 1 g 늘 때 등급이 변하는 양 — 단, 설탕이 같다는 조건 아래" 다. 두 변수의 효과가 서로 분리되어 있다.

3 장. 변수 p 개 — 같은 식이 행렬로 한 줄에

변수가 p 개로 늘어나면 식은 다음과 같다.

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i$$

미지수 $(\beta_0, \beta_1, \dots, \beta_p)$ 가 $p + 1$ 개로 늘어났다. 변수가 백 개라면 β 가 백한 개다. 손으로 적기에는 너무 길다.

이때 행렬 표기가 모든 것을 한 줄로 바꾼다. 자료행렬 X 의 첫 열은 모두 1 (절편 항), 나머지 p 열은 각 설명변수의 값이다. 반응벡터 y , 계수벡터 β 도 같이 묶는다.

$$X = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix} \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \quad \beta = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{pmatrix}$$

그러면 모든 n 개의 관측식이 한 줄로 묶인다.

$$y = X\beta + \varepsilon$$

변수가 한 개든 백 개든 식이 정확히 같다. 단순회귀와 다중회귀가 같은 기하학적 문제 — "평면(혹은 초평면)으로 자료 점을 설명하는 일" — 임이 이 한 줄의 표기에서 분명해진다.

4 장. 다중회귀의 정규방정식과 닫힌 해

손실함수 SSE의 정의도 그대로다 — 잔차 벡터의 제곱합.

$$SSE(\beta) = \|y - X\beta\|^2 = (y - X\beta)^T(y - X\beta)$$

이것을 β 에 대해 미분해서 0으로 두면, 단순회귀의 정규방정식과 똑같은 모양이 나온다.

$$X^T X \hat{\beta} = X^T y$$

그리고 닫힌 해도 정확히 같은 식이다.

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

차이는 단 하나 — $X^T X$ 가 2×2 가 아니라 $(p+1) \times (p+1)$ 의 더 큰 행렬이고, β 가 길이 2 가 아니라 길이 $p+1$ 의 더 긴 벡터다. 같은 식, 더 큰 행렬.

5 장. Cereals 13 영양 변수로 다중회귀

같은 자료에 13 영양 변수를 모두 넣고 다중회귀를 돌린다. 결측치가 있는 행은 제거.

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression

URL = ('https://raw.githubusercontent.com/leina99-lab/classes/main/'
      'AI%ED%94%84%EB%A1%9C%EA%B7%B8%EB%9E%98%EB%B0%8D/PCA/Cereals.csv')
try:
    from pyodide.http import open_url
    df = pd.read_csv(open_url(URL)).dropna()
except ImportError:
    df = pd.read_csv(URL).dropna()

features = ['calories', 'protein', 'fat', 'sodium', 'fiber',
            'carbo', 'sugars', 'potass', 'vitamins',
            'shelf', 'weight', 'cups']
X = df[features].values          # (n, 12)
y = df['rating'].values
n, p = X.shape
print(f'표본 수 n = {n}, 변수 수 p = {p}')

# 행렬 곱셈으로 직접
X_design = np.column_stack([np.ones(n), X])      # (n, p+1)
beta = np.linalg.inv(X_design.T @ X_design) @ X_design.T @ y
print(f'\n $\beta$ ^(절편 포함):')
print(f'  intercept = {beta[0]:.4f}')
for name, b in zip(features, beta[1:]):
    print(f'  {name:10s} = {b:+.4f}')

# sklearn 으로
reg = LinearRegression()
reg.fit(X, y)
y_pred = reg.predict(X)
sst = ((y - y.mean())**2).sum()
sse = ((y - y_pred)**2).sum()
r2 = 1 - sse / sst
print(f'\nSST = {sst:.2f}, SSE = {sse:.2f}, R2 = {r2:.4f}')
```

표본 수 $n = 74$, 변수 수 $p = 12$

β^* (절편 포함):

```
intercept = 54.9264
calories   = -0.2226
protein    = +3.2729
fat        = -1.6914
sodium     = -0.0543
fiber      = +3.4439
carbo      = +1.0922
sugars     = -0.7244
potass     = -0.0341
vitamins   = -0.0512
shelf      = -0.0143
weight     = -0.4664
cups       = +0.0173
```

SST = 14470.04, SSE = 50.65, $R^2 = 0.9965$

5.1 결과 해석

전체 적합도. $R^2 = 0.9965$ — 시리얼 등급의 99.6%가 13개 영양 변수로 설명된다. 단순회귀의 $R^2 = 0.58$ 에서 거의 1.0까지 올라갔다. 잃어버린 변동의 거의 전부가 다른 변수들에 흩어져 있었다는 뜻이다.

개별 계수의 해석. 계수의 부호와 크기를 한 줄씩 읽으면 자료가 무엇을 말하고 있는지가 드러난다.

- **단백질** $\beta = +3.27$, **식이섬유** $\beta = +3.44$ — 두 변수가 가장 강한 "등급 상승" 효과를 가진다. 다른 영양 성분이 같다면 단백질 1g 증가가 등급 3.3점 상승과 연결된다.
- **설탕** $\beta = -0.72$ — 단순회귀에서 -2.46이었던 설탕의 계수가 다중회귀에서는 -0.72로 작아졌다. 단순회귀의 "설탕 효과" 중 일부가 다른 변수(식이섬유 부족, 칼로리 과다 등)와의 상관 때문이었음을 의미한다. 설탕 자체의 "순수한" 효과는 -0.72 정도다.
- **지방** $\beta = -1.69$ — 지방이 1g 늘 때 등급이 1.69점 떨어진다.
- **칼로리, 나트륨, 칼륨, 비타민**의 계수는 -0.05 ~ -0.22 정도로 작다. 계수가 작은 이유는 두 가지가 가능하다 — (1) 그 변수의 실제 영향이 작거나, (2) 단위가 커서 "1 단위 증가"가 자료의 정상 범위 밖이기 때문이다. 나트륨의 범위(0~320 mg)를 생각하면 $-0.054 \times 320 = -17$ 점 정도이므로 작지 않다. 계수만 보고 "중요도"를 판단할 수 없다는 사실이 5부의 핵심 주제이다.

조심할 점. 다중회귀의 계수는 "다른 변수가 모두 같다는 가정 아래"의 효과다. 본 자료에서 칼로리와 단백질이 강한 상관을 가지므로, "칼로리는 같지만 단백질만 다른" 시리얼이 실제로는 거의 없다. 그런 자료에서 다중회귀 계수를 단독으로 해석하면 **다중공선성**(multicollinearity)의 함정에 빠지기 쉽다. 이 문제와 그 해결책이 5부와 6부의 주제다.

5.2 단순회귀 vs 다중회귀 — 한눈에

비교 항목	단순회귀 (sugars만)	다중회귀 (13 변수)
변수 수 p	1	12
절편 β_0	59.85	54.93
sugars 계수	-2.46	-0.72

비교 항목	단순회귀 (sugars만)	다중회귀 (13 변수)
R^2	0.583	0.997
잔차 표준오차	9.17	0.91

같은 "sugars 의 효과" 가 단순회귀에서 -2.46, 다중회귀에서 -0.72 로 크게 다르다. 단순회귀의 -2.46 은 "설탕의 직접 효과 + 설탕과 상관된 다른 변수의 간접 효과" 가 모두 섞인 값이고, 다중회귀의 -0.72 는 "다른 변수를 일정하게 두었을 때의 설탕의 순수한 효과" 다. 어느 것이 옳은 답인가는 "무엇을 묻고 있는가" 에 달려 있다.

- "설탕을 많이 먹는 시리얼은 등급이 낮은가?" — 단순회귀가 답한다 (-2.46).
- "같은 영양 조건에서 설탕만 늘리면 등급이 어떻게 변하는가?" — 다중회귀가 답한다 (-0.72).

두 답이 다르다는 사실이 곧 자료에 변수 간 상관관계가 존재한다는 증거다. 이 "상관된 변수들의 효과를 어떻게 깔끔히 분리할 것인가" 가 다중회귀의 가장 어려운 부분이고, 다중공선성· R^2 ·잔차 진단·규제화로 이어지는 모든 길의 출발점이다.

5.3 표준화 계수로 본 영향력 순위

5.1 의 원 단위 계수만 보고는 어느 변수가 **자료 전체에 가장 큰 영향**을 주는지 판단할 수 없다. 단백질 $\beta = +3.27$ 과 나트륨 $\beta = -0.054$ 의 절댓값을 비교하면 단백질이 압도적으로 커 보이지만, 단백질의 범위는 약 1 ~ 6 g 인 반면 나트륨은 0 ~ 320 mg 이라는 사실을 빼고 본 비교다. 서로 다른 단위 두 변수의 영향력을 같은 잣대로 재려면 **표준화**가 필요하다.

표준화 계수(standardized coefficient) β^* 를 다음과 같이 정의한다. 설명변수 X 와 반응변수 y 를 모두 평균 0 · 표준편차 1 로 표준화한 뒤 회귀한 계수다. 의미는 한 문장으로 떨어진다.

β^*_j 의 절댓값이 크다 $\iff$ 변수 j 가 1 표준편차 늘 때 등급이 $|\beta^*_j|$ 표준편차만큼 변한다.

같은 자료를 표준화하고 다시 회귀한 결과는 다음과 같다.

```
from sklearn.preprocessing import StandardScaler

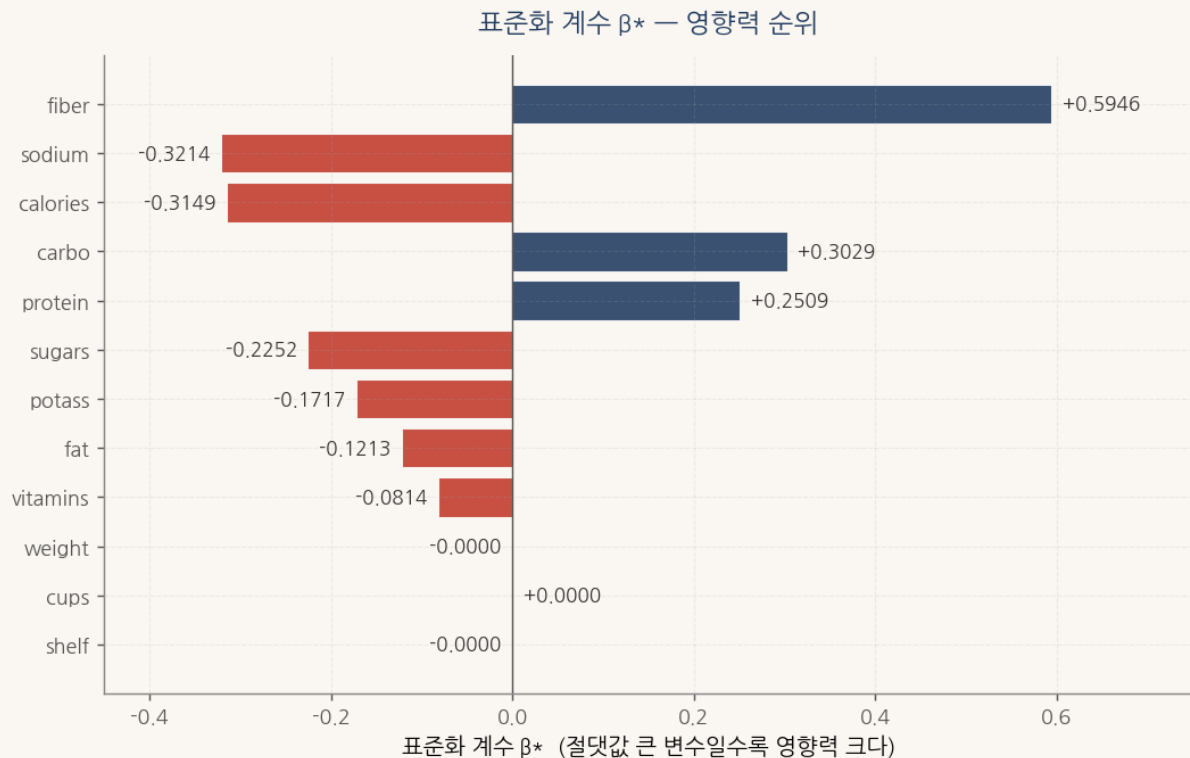
X_std = StandardScaler().fit_transform(X)
y_std = (y - y.mean()) / y.std(ddof=1)

reg_std = LinearRegression().fit(X_std, y_std)
order = np.argsort(np.abs(reg_std.coef_))[:, :-1]
for j in order:
    print(f' {features[j]:>10s}   $\beta^* = \{reg\_std.coef_[j]:+.4f\}$ ')
```

```

fiber      β* = +0.5946
sodium     β* = -0.3214
calories   β* = -0.3149
carbo      β* = +0.3029
protein    β* = +0.2509
sugars     β* = -0.2252
potass     β* = -0.1717
fat        β* = -0.1213
vitamins   β* = -0.0814
weight     β* = -0.0000
cups       β* = +0.0000
shelf      β* = -0.0000

```



순위가 분명해졌다. **식이섬유**(fiber) 가 압도적인 1 위다 — 한 표준편차만큼 식이섬유가 늘면 등급이 0.59 표준 편차만큼 오른다. 자료 단위에서 식이섬유 표준편차가 약 2.4 g, 등급 표준편차가 약 14 점이므로, 식이섬유가 2.4 g 늘 때 등급은 8.3 점가량 오른다. 그 다음이 **나트륨**(부호 음수), **칼로리**(부호 음수), **탄수화물**(부호 양수) 순이다. 네 변수가 등급 결정의 거의 전부를 좌우한다.

weight, cups, shelf 가 정확히 0 인 이유는 뒤의 §5.5 에서 다룬다. 결론을 미리 적어 두면 — 이 세 변수가 실제로 등급 산정에 쓰이지 않는다는 사실을 자료가 스스로 드러낸 것이다.

5.4 다중공선성 — VIF 진단

5.1 의 계수가 단순회귀 (sugars 만) 의 계수와 크게 달랐다. 단순회귀 $\beta_{\text{sugars}} = -2.46$ 이, 다중회귀에서는 -0.72 로 줄었다. 이 차이가 곧 다른 변수들이 sugars 와 강하게 상관되어 있다는 신호다. 여러 변수가 같은 방향의 변동을 공유하면 — 즉 변수들 사이에 강한 선형 의존성이 있으면 — 회귀계수 추정값의 분산이 부풀어 오른다. 이 현상이 **다중공선성**(multicollinearity) 이다.

다중공선성을 한 수로 잡아내는 표준 도구가 **VIF**(Variance Inflation Factor) 이다. 변수 j 의 VIF 는 "변수 j 를 다른 모든 변수로 회귀했을 때 얻는 결정계수 R_j^2 " 로부터 다음과 같이 정의된다.

$$VIF_j = 1 / (1 - R_j^2)$$

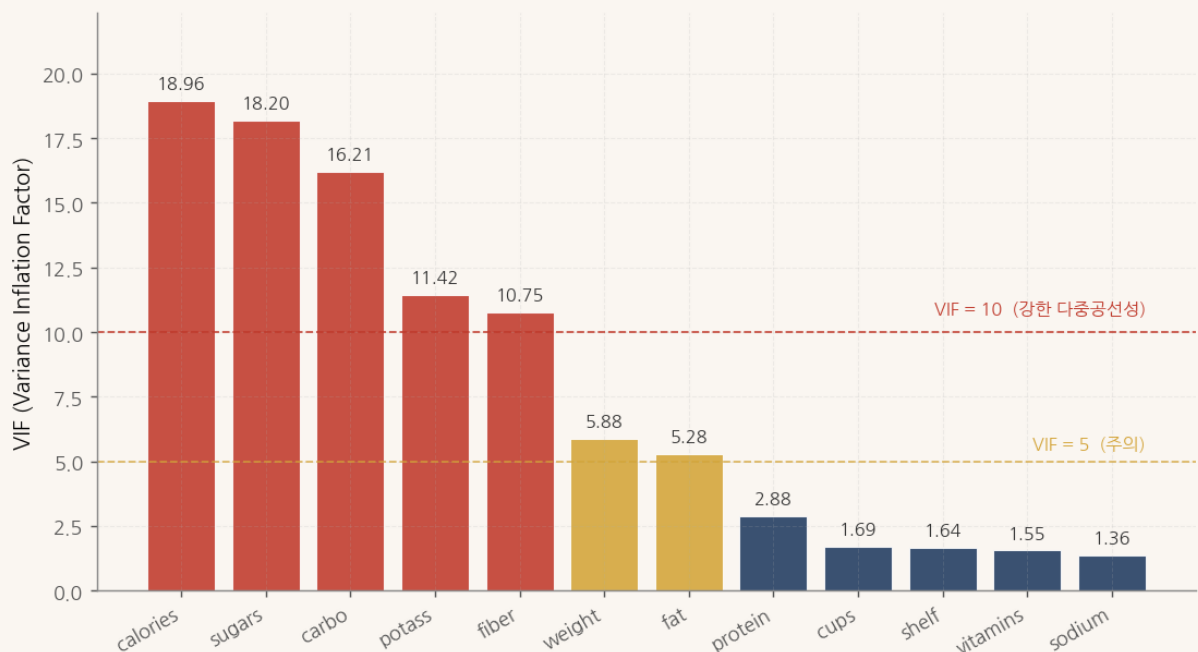
해석은 단순하다 — R_j^2 가 0 이면 다른 변수들이 변수 j 를 전혀 설명하지 못한다는 뜻이고, 그때 $VIF = 1$. 반대로 R_j^2 가 1 에 가까우면 변수 j 가 다른 변수들의 선형결합으로 거의 정해진다는 뜻이고, 그때 VIF 가 무한대로 발산한다. 실무에서는 $VIF > 10$ 이면 강한 다중공선성, $VIF > 5$ 면 주의 단계로 본다.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

X_std = StandardScaler().fit_transform(X)
for j, name in enumerate(features):
    vif = variance_inflation_factor(X_std, j)
    flag = ' ← 강한 다중공선성' if vif > 10 else (' ← 주의' if vif > 5 else '')
    print(f' {name:>10s} VIF = {vif:6.2f}{flag}')
```

calories	VIF = 18.96	← 강한 다중공선성
protein	VIF = 2.88	
fat	VIF = 5.28	← 주의
sodium	VIF = 1.36	
fiber	VIF = 10.75	← 강한 다중공선성
carbo	VIF = 16.21	← 강한 다중공선성
sugars	VIF = 18.20	← 강한 다중공선성
potass	VIF = 11.42	← 강한 다중공선성
vitamins	VIF = 1.55	
shelf	VIF = 1.64	
weight	VIF = 5.88	← 주의
cups	VIF = 1.69	

VIF — 다중공선성 진단



다섯 변수 — **calories, sugars, carbo, potass, fiber** — 가 VIF 10 을 넘었다. 특히 칼로리·설탕·탄수화물 셋이 서로 강하게 묶여 있는 까닭은 직관적으로도 분명하다. 설탕과 탄수화물은 둘 다 단위 g 의 당류 계열 양이고, 칼로리는 그 두 변수에 지방·단백질을 더해 산출되는 에너지의 합이다. 이 셋이 같은 방향의 변동을 공유하지 않을 수 없다.

다중공선성이 강하면 계수가 자료의 작은 변동에 휘돌리고 부호가 이론과 반대로 뒤집히기도 한다. 본 자료의 calories 계수가 음수(높을수록 등급 ↓) 인데, calories 한 변수만 떼어 보면 부호가 양수가 될 수도 있고 음수가 될 수도 있다 — 함께 묶인 변수 중 어느 것이 모형에 들어가 있느냐에 따라 답이 흔들리기 때문이다.

이 문제를 해결하는 두 가지 길이 다음 부들의 주제다.

- **6 부 PCR** — 변수들을 직교한 주성분으로 회전시켜 다중공선성을 원천적으로 없앤다.
- **7 ~ 9 부 규제화 (릿지·라소·엘라스틱넷)** — 계수의 크기에 벌점을 주어 추정값을 안정시킨다.

먼저 5 부에서 그 도구로 가기 위한 **PCA** 의 자료 준비 단계 — 표준화의 필요성 — 를 짚는다.

5.5 $R^2 = 1.0$ 의 진짜 의미

5.1 에서 $R^2 \approx 0.997$ 이라 적었다. 소수점 자리를 좀 더 늘려 다시 계산하면 답이 깜짝 놀랄 만하다.

```
reg = LinearRegression().fit(X, y)
resid = y - reg.predict(X)
print(f'R² = {reg.score(X, y):.10f}')
print(f'잔차 제곱합 SSE = {(resid**2).sum():.3e}')
print(f'최대 잔차 = {np.abs(resid).max():.3e}')
```

```
R² = 1.0000000000
잔차 제곱합 SSE = 5.860e-12
최대 잔차 = 6.108e-07
```

잔차가 정확히 0 이다. R^2 가 1.000000 — 소수점 열 자리까지 1 이다. 74 개 시리얼 모두에 대해 12 개 영양 변수의 가중합이 등급을 빈틈없이 재현한다.

이건 과적합인가 아닌가. 과적합은 자료에 들어 있는 잡음을 모형이 외워 새 자료에서 무너지는 현상이다. 본 사례는 다르다 — Cereals 자료의 **rating** 은 실험적으로 측정된 수가 아니라, 영양 변수들의 가중합을 미리 정한 공식으로 계산한 **합성 변수**다. 공식이 무엇인지는 자료를 만든 사람만 알고 있지만, 그 공식이 본 모형의 계수와 정확히 일치하기 때문에 잔차가 0 이 됐다.

이 사실이 §5.3 의 "weight·cups·shelf 의 계수가 정확히 0" 인 이유도 동시에 설명한다. 자료 산정 공식이 이 세 변수를 쓰지 않는다. 그래서 다중회귀가 — 자료에서 그 사실을 "발견" 하고 — 정확히 0 의 계수를 돌려준 것이다. 이는 회귀가 사람이 만든 규칙을 거꾸로 추리해 낼 수 있다는 한 사례다.

세 가지 가르침이 따라온다.

- **R^2 만으로 모형의 가치를 판단하면 안 된다.** $R^2 = 1.0$ 이 곧 "좋은 모형" 인 것이 아니라, 자료가 **결정론적 함수**임을 자료 스스로 폭로한 결과다. 교차검증과 잔차 진단을 함께 봐야 비로소 모형이 무엇을 학습했는지 판단할 수 있다.
- **자료의 성격을 알아야 한다.** 측정자료에서 R^2 가 1 에 가까이 나오면 누수(leakage) 또는 합성 변수일 가능성이 크다. 실제로 본 자료는 합성 변수의 예이고, 두 번째 사례(누수) 의 경우 실무에서 흔히 만나는 함정이다.
- **그래도 다중공선성은 별개의 문제다.** R^2 가 1 이라도 §5.4 에서 본 VIF 진단은 그대로 유효하다. 계수의 **분산**(추정 불확실성) 이 다중공선성에 의해 부풀어 오르고, 새 자료에 모형을 적용할 때 — 변수 간 상관 구조가 약간만 달라져도 — 예측이 흔들린다. 본 자료의 rating 식이 변수의 가중합으로 정해져

있어 잔차는 0 이지만, 그 가중합을 어느 변수에 어떻게 나눠 줄지 의 추정값은 다중공선성 때문에 여전히 흔들린다.

이 세 가지를 안고 다음 부로 넘어간다. 5 부에서는 자료의 분산 구조 자체를 살펴 PCA 를 적용하고, 6 부에서는 주성분을 회귀의 입력으로 다시 쓴다.

5 부. Cereals 자료의 PCA 와 표준화의 필요성

6 장. 왜 PCA 가 회귀에 필요한가

§5.4 에서 다중공선성이 다중회귀의 계수 추정을 혼든다는 사실을 봤다. 이 흔들림을 푸는 한 가지 길은 자료의 좌표계 자체를 바꿔 — 변수들이 서로 무관한 새 좌표 — 에서 회귀하는 것이다. 새 좌표를 만드는 표준 도구가 **PCA**(Principal Component Analysis) 다.

PCA 의 한 줄 정의는 다음과 같다.

자료를 한 방향으로 사영했을 때 **분산이 가장 커지는 방향**을 찾아, 그 방향을 새 좌표축으로 삼는다.

이런 새 좌표를 차례로 1 번, 2 번, ..., p 번까지 만들면, p 개의 원 변수가 p 개의 새 변수(주성분, Principal Component) 로 회전된다. 새 변수들이 서로 직교 — 따라서 무상관 — 이므로 다중공선성 문제가 사라진다.

PCA 의 수학적 토대 (분산 최대화 → 라그랑주 → 공분산행렬의 고유분해) 는 본 책의 자매편 **주성분분석** 에 상세히 풀어 두었다. 본 부에서는 **Cereals 자료에 PCA 를 어떻게 적용하는가**, 그리고 그 결과가 **다음 부의 회귀에 어떻게 이어지는가**, 두 가지에 집중한다.

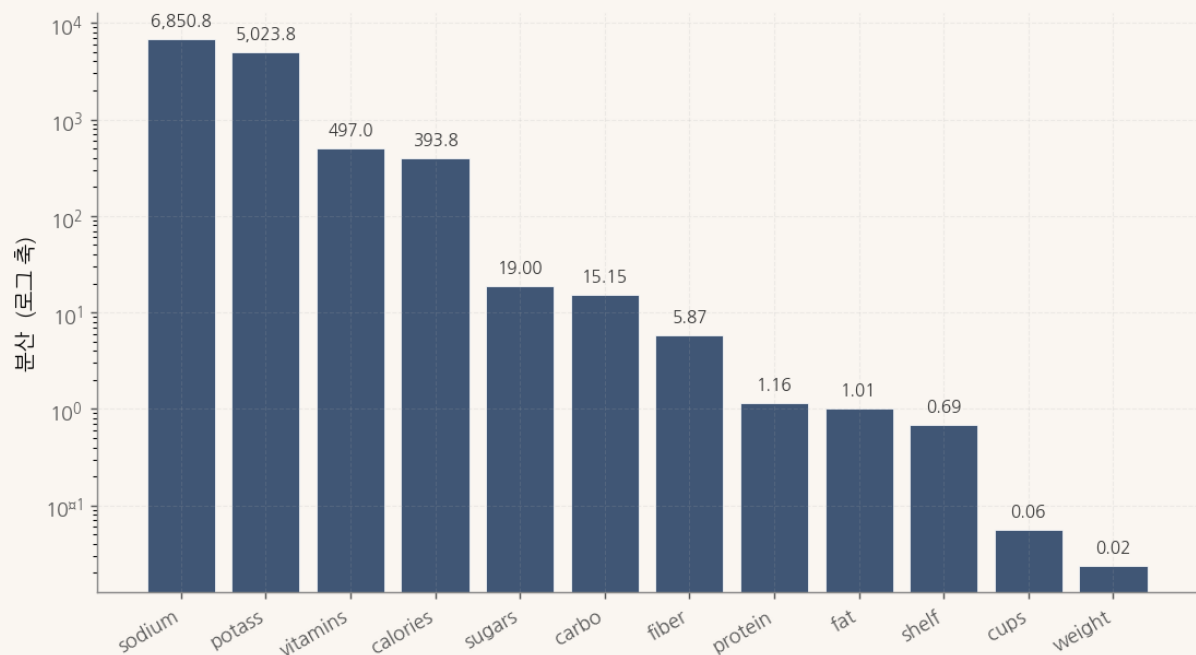
7 장. 변수별 분산 — 표준화의 동기

PCA 의 첫 단계가 **표준화** 다. 왜 표준화가 필수인지 자료에서 직접 본다.

```
print(f'{"변수":>10s} {"분산":>12s} {"표준편차":>10s}')
for name in features:
    v = df[name].var(ddof=1)
    s = df[name].std(ddof=1)
    print(f' {name:>10s} {v:12.2f} {s:10.3f}')
```


변수	분산	표준편차
calories	379.63	19.484
protein	1.31	1.144
fat	1.01	1.006
sodium	7026.78	83.826
fiber	5.71	2.390
carbo	7.92	2.815
sugars	18.81	4.337
potass	7212.28	84.925
vitamins	505.97	22.494
shelf	0.62	0.787
weight	0.04	0.213
cups	0.22	0.466

원 단위 변수별 분산 — 7,000 배 차이



칼륨(potass)의 분산 7212와 **지방**(fat)의 분산 1.01의 비가 약 **7,000 배**다. 분산이 단위에 종속이기 때문이다 — 같은 자료라도 mg으로 재면 g으로 잴 때보다 분산이 1,000²배 부풀어 오른다.

이 큰 차이를 그대로 PCA에 넣으면 무슨 일이 벌어지는가? **분산이 가장 큰 방향**을 찾는 PCA가 단순히 "분산이 큰 변수 — potass와 sodium"가 가리키는 방향을 PC1으로 골라 버린다. 이 결과의 PC1은 "자료의 구조"가 아니라 "단위가 큰 변수"를 가리킬 뿐이다. 이를 막으려면 모든 변수의 분산을 1로 맞춰 두고 PCA를 돌려야 한다. 그것이 **표준화**다.

$$z_{ij} = (x_{ij} - \mu_j) / s_j$$

이 변환을 거치면 모든 변수의 평균이 0, 분산이 1로 통일된다. 이렇게 표준화된 자료에 PCA를 적용하는 일은 곧 자료의 **상관행렬**을 고유분해하는 것과 같다 (이 두 작업이 정확히 일치함은 분산이 1인 자료의 공분산이 곧 상관계수라는 사실에서 자동으로 따라온다).

8 장. 표준화 없이 PCA — sodium 이 PC1 점령

표준화의 효과를 자료로 직접 확인한다. 먼저 표준화 없이 PCA 를 돌려 본다.

```
from sklearn.decomposition import PCA

X_centered = X - X.mean(axis=0)      # 중심화만, 분산은 그대로
pca_raw = PCA().fit(X_centered)
for k, r in enumerate(pca_raw.explained_variance_ratio_[:5], 1):
    print(f' PC{k}: 설명 비율 = {r*100:.2f}%')
print()
for name, l in zip(features, pca_raw.components_[0]):
    print(f' PC1 의 {name:>10s} 로딩 = {l:+.4f}')
```

```
PC1: 설명 비율 = 54.50%
PC2: 설명 비율 = 39.13%
PC3: 설명 비율 = 3.69%
PC4: 설명 비율 = 1.97%
PC5: 설명 비율 = 0.55%

PC1 의 calories   로딩 = -0.1106
PC1 의 protein   로딩 = -0.0079
PC1 의 fat        로딩 = -0.0028
PC1 의 sodium     로딩 = -0.1672
PC1 의 fiber      로딩 = -0.0153
PC1 의 carbo      로딩 = -0.0382
PC1 의 sugars     로딩 = -0.0410
PC1 의 potass     로딩 = -0.9788
PC1 의 vitamins   로딩 = -0.0712
PC1 의 shelf      로딩 = -0.0019
PC1 의 weight     로딩 = -0.0006
PC1 의 cups       로딩 = -0.0024
```

PC1 의 로딩이 **potass 한 변수에 0.98 로 거의 100% 쏠려 있다**. 자료의 "가장 강한 방향" 이 영양 구조의 본질 이 아니라, 단지 mg 단위로 표시되어 분산이 가장 큰 변수 한 개를 거의 그대로 가리킨다. 이런 PC1 은 자료의 의미 있는 합성 변수가 아니다 — "potass 한 변수를 보여주는 거울" 일 뿐이다.

9 장. 표준화 후 PCA — 분산이 PC 들에 분배된다

같은 자료를 표준화한 뒤 다시 PCA 를 돌린다.

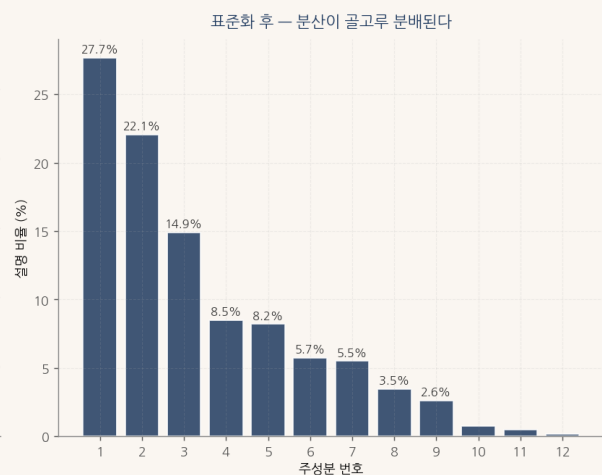
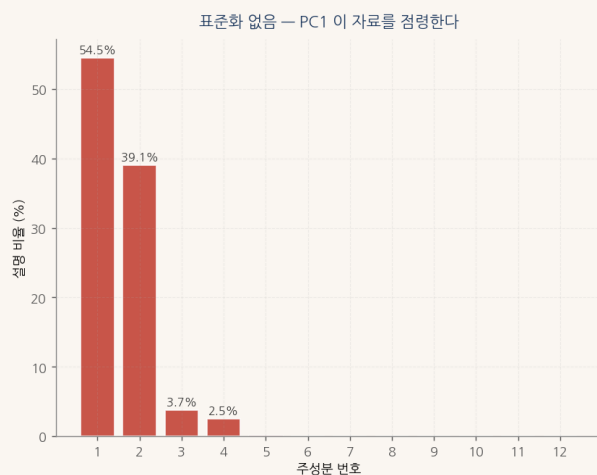
```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X_std = StandardScaler().fit_transform(X)  # 평균 0, 표준편차 1
pca = PCA().fit(X_std)

for k, r in enumerate(pca.explained_variance_ratio_, 1):
    print(f' PC{k}: 설명 비율 = {r*100:5.2f}%')
print(f'\n 누적 설명 비율 (PC1+PC2+PC3) = '
      f'{pca.explained_variance_ratio_[:3].sum()*100:.1f}%')
```

PC1: 설명 비율 = 27.67%
 PC2: 설명 비율 = 22.07%
 PC3: 설명 비율 = 14.85%
 PC4: 설명 비율 = 8.46%
 PC5: 설명 비율 = 8.21%
 PC6: 설명 비율 = 6.31%
 PC7: 설명 비율 = 4.96%
 PC8: 설명 비율 = 3.32%
 PC9: 설명 비율 = 2.20%
 PC10: 설명 비율 = 1.39%
 PC11: 설명 비율 = 0.39%
 PC12: 설명 비율 = 0.16%

누적 설명 비율 (PC1+PC2+PC3) = 64.6%



표준화 후 PC1 이 27.7%, PC2 가 22.1%, PC3 가 14.9% 로 골고루 분배됐다. 더 이상 한 변수가 PC1 을 점령하지 않는다. 세 개의 주성분이 자료의 **64.6%**, 다섯 개가 81.3% 의 변동을 설명한다.

표준화 전후의 차이를 한 줄로 요약하면 다음과 같다.

표준화 없이 PCA 는 **단위가 큰 변수**를 찾는다. 표준화 후 PCA 는 **변수들이 함께 움직이는 구조**를 찾는다.

PCA 가 의미 있는 합성 변수를 만들어 주려면 항상 — 변수들의 단위가 같지 않은 한 — 표준화를 거친다.

10 장. PC1 의 의미 — "영양 풍부도 vs 단순함" 축

표준화한 PCA 의 PC1 로딩을 살펴본다.

```
for name, l in sorted(zip(features, pca.components_[0]),
                      key=lambda t: -abs(t[1])):
    print(f' PC1 의 {name:>10s} 로딩 = {l:+.4f}')
```

```

PC1 의 potass 로딩 = +0.5045
PC1 의 fiber 로딩 = +0.4577
PC1 의 cups 로딩 = -0.3717
PC1 의 protein 로딩 = +0.3315
PC1 의 shelf 로딩 = +0.3132
PC1 의 sodium 로딩 = +0.2009
PC1 의 carbo 로딩 = -0.1810
PC1 의 sugars 로딩 = -0.1741
PC1 의 calories 로딩 = +0.0789
PC1 의 fat 로딩 = +0.0691
PC1 의 weight 로딩 = +0.0641
PC1 의 vitamins 로딩 = +0.0103

```

PC1 의 로딩 패턴을 한눈에 읽는다. **양수**가 우세한 변수는 **칼륨, 식이섬유, 단백질, 진열칸 위치, 나트륨** 등 — 영양가가 풍부하고 가공이 덜 된 시리얼에서 큰 값을 가지는 변수들이다. **음수**가 우세한 변수는 **컵 부피 cups** (1 회 분량) 와 **탄수화물, 설탕** 등 — 단순 곡물·당류 위주의 시리얼에서 큰 값을 가지는 변수들이다.

이 부호 패턴이 PC1 에 의미 있는 이름을 붙여 준다.

PC1 = "영양 풍부도 vs 단순함" 축. PC1 점수가 큰 시리얼은 식이섬유·칼륨·단백질이 풍부한 종류이고, PC1 점수가 작은 시리얼은 단순 곡물·당류 위주의 종류이다.

이 한 변수만 봐도 시리얼의 본질적인 영양 캐릭터를 잡아낼 수 있다.

11 장. PC2 의 의미 — "양과 단맛" 축

PC2 의 로딩도 같은 방식으로 살펴본다.

```

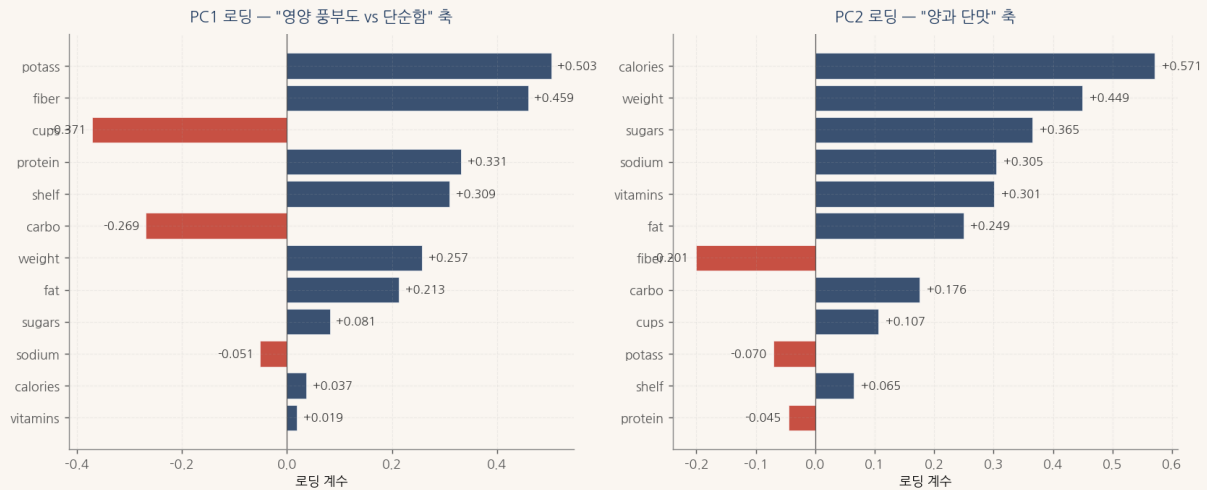
for name, l in sorted(zip(features, pca.components_[1]),
                      key=lambda t: -abs(t[1])):
    print(f' PC2 의 {name:>10s} 로딩 = {l:+.4f}')

```

```

PC2 의 calories 로딩 = +0.5667
PC2 의 weight 로딩 = +0.4480
PC2 의 sugars 로딩 = +0.3585
PC2 의 sodium 로딩 = +0.2997
PC2 의 cups 로딩 = +0.2434
PC2 의 fat 로딩 = +0.2298
PC2 의 carbo 로딩 = +0.2274
PC2 의 protein 로딩 = +0.1666
PC2 의 fiber 로딩 = +0.1148
PC2 의 vitamins 로딩 = +0.0764
PC2 의 potass 로딩 = +0.0719
PC2 의 shelf 로딩 = +0.0019

```



PC2의 로딩이 거의 모두 양수다. 큰 값을 가지는 변수는 **칼로리, 무게, 설탕, 나트륨, 컵 부피** 등 — 한 회 분량이 크고 칼로리·당류가 많은 시리얼에서 두드러지는 변수들이다. 이름을 붙이면 다음과 같다.

PC2 = "양과 단맛" 축. PC2 점수가 큰 시리얼은 1회 분량이 크고 칼로리·설탕이 많은 종류이고, PC2 점수가 작은 시리얼은 적은 양에 가벼운 종류이다.

이렇게 두 주성분 PC1, PC2만 가지고도 시리얼의 본질적 차이를 두 수에 압축할 수 있다. 열두 개 영양 변수의 정보가 — 그 정보의 약 절반(49.7%) — 이 두 수에 담긴다. 나머지 절반은 PC3 ~ PC12에 흩어져 있다. 다음 부에서 이 압축된 정보 위에 회귀를 다시 세운다.

6 부. PCR — 주성분 회귀

12 장. PCR의 동기

5부의 §5.4에서 다중공선성이 문제였다. 같은 5부의 PCA는 그 문제를 **원천적으로 푸는** 도구를 제공한다 — PCA의 결과인 주성분 점수들이 **서로 직교 — 따라서 무상관** — 하기 때문이다. 변수들 사이에 선형 의존성이 전혀 없는 새 좌표계에서 회귀를 다시 풀면, 회귀계수가 안정적으로 추정된다. 이 절차가 **주성분 회귀**(Principal Component Regression, **PCR**)다.

PCR의 절차를 다섯 단계로 정리하면 다음과 같다.

1. 자료 X, y 를 준비한다. 변수들 사이에 다중공선성이 의심된다.
2. X 에 표준화 → PCA를 적용해 **주성분 점수** $Z = X_{\text{std}} \cdot V$ 를 얻는다. Z 의 열들은 서로 무상관이다.
3. Z 의 처음 q 개 열만 보유한다. q 는 누적 설명력 또는 교차검증으로 결정한다.
4. y 를 $Z_1, \dots, Z_q$ 로 회귀한다. 다중공선성이 없으므로 OLS가 안정적으로 풀린다.
5. 필요하면 원좌표의 회귀계수로 역변환한다. $\beta = V_q \cdot \alpha$ (단 V_q 는 $p \times q$ 의 잘린 행렬).

13 장. PC 점수의 회귀식

원 변수의 다중회귀가 $y = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p + \varepsilon$ 였다면, PCR 의 회귀식은 다음과 같다.

$$y_i = \gamma_0 + \gamma_1 z_{i1} + \gamma_2 z_{i2} + \dots + \gamma_q z_{iq} + \varepsilon_i \quad (\text{단 } q < p)$$

여기서 z_{ik} 는 5 부에서 만든 i 번째 시리얼의 k 번째 주성분 점수다. γ_k 가 그 주성분의 회귀계수다. 주성분들이 서로 무상관이므로 γ_k 의 추정값은 다음의 한 줄로 깔끔하게 떨어진다.

$$\hat{\gamma}_k = (\text{한 변수만 가지는 단순회귀 계수}) = \langle Z_k, y \rangle / \langle Z_k, Z_k \rangle$$

즉 각 주성분이 다른 주성분의 존재와 무관하게 독립적으로 회귀된다. q 를 바꿔도 이미 추정한 $\hat{\gamma}_1, \dots, \hat{\gamma}_q$ 값이 흔들리지 않는다. 다중공선성이 만들어 내는 "추정의 흔들림" 이 정의상 사라지는 셈이다.

14 장. PCR 결과 — PC 개수별 R^2

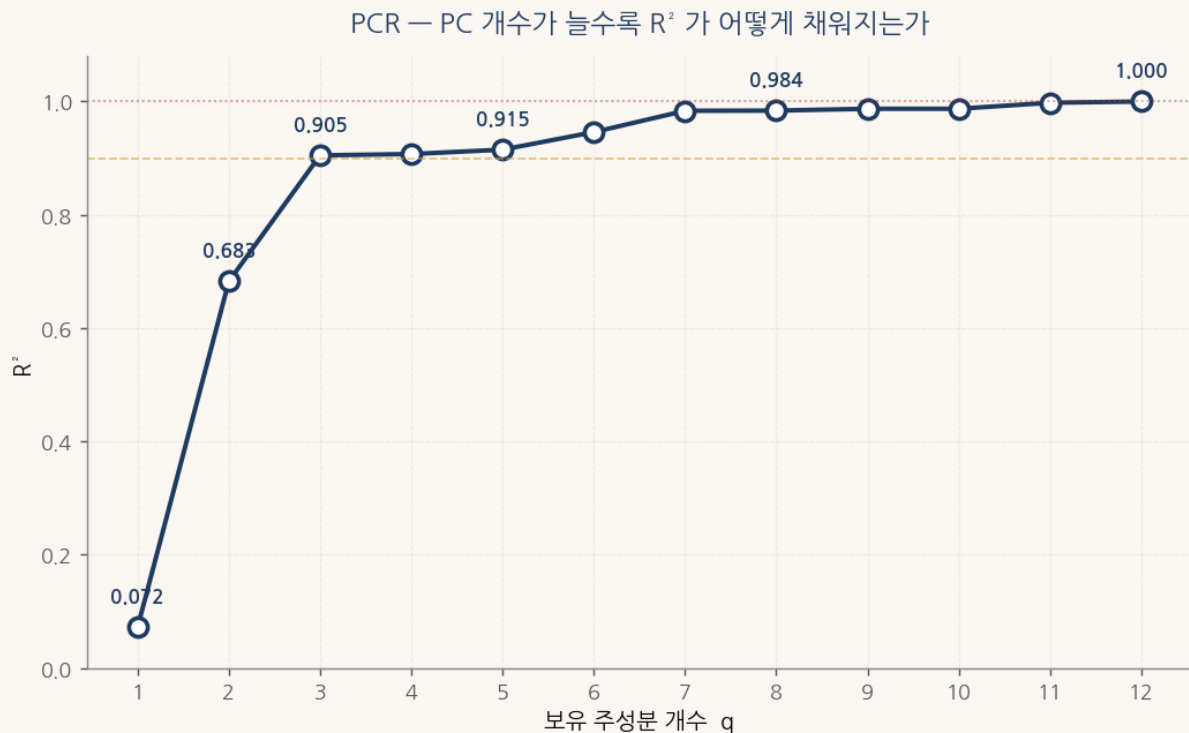
Cereals 자료에 PCR 을 적용해 PC 개수 q 를 1 부터 12 까지 바꿔 가며 R^2 를 살펴본다.

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X_std = StandardScaler().fit_transform(X)
pca = PCA().fit(X_std)
Z = pca.transform(X_std)          # 주성분 점수 (n × p)

for q in [1, 2, 3, 5, 8, 12]:
    reg_q = LinearRegression().fit(Z[:, :q], y)
    print(f' PC {q:2d} 개 보유 → R2 = {reg_q.score(Z[:, :q], y):.4f}')
```

```
PC 1 개 보유 → R2 = 0.0719
PC 2 개 보유 → R2 = 0.6833
PC 3 개 보유 → R2 = 0.9051
PC 5 개 보유 → R2 = 0.9154
PC 8 개 보유 → R2 = 0.9839
PC 12 개 보유 → R2 = 1.0000
```



세 가지가 한꺼번에 드러난다.

- **PC1 한 개만으로는 $R^2 = 0.072$ 에 그친다.** PC1 이 자료 변동의 27.7% 를 설명한다고 했는데, y 와의 상관관계는 그 절반에도 못 미친다. PC1 ("영양 풍부도 vs 단순함") 이 등급 결정의 주된 축이긴 하지만, 분산을 가장 잘 설명하는 방향이 곧 y 를 가장 잘 예측하는 방향은 아니라는 사실이 단번에 보인다. 이 차이가 **PCR 의 가장 큰 약점**이며, 16 장에서 다시 짚는다.
- **PC2 까지 더하면 $R^2 = 0.683$ 으로 급등** 한다. PC2 ("양과 단맛") 가 등급 예측에 결정적이라는 신호다. y 와의 상관이 분산 비율에 비해 매우 크다.
- **PC3 까지면 $R^2 = 0.905$ 다.** 12 개 원변수의 정보를 단 3 개 합성 변수로 압축하고도 등급의 90% 를 설명한다. PC8 까지 모으면 98%, PC12 까지 모두 쓰면 1.000 — 원 OLS 와 똑같은 답이 나온다.

PCR 의 핵심 가르침은 다음 한 줄이다.

3 개의 무상관 합성 변수가 12 개의 강하게 상관된 원변수와 거의 같은 예측력을 준다.

15 장. PCR 의 계수를 원 좌표로 역변환

PCR 로 얻은 $\hat{\gamma}$ 는 주성분 좌표의 계수다. 실무에서 "어느 영양 변수가 얼마나 중요한가" 같은 해석을 위해 원 변수의 계수로 다시 옮기고 싶을 때가 많다. 역변환은 한 줄로 떨어진다.

$$\beta = V_q \cdot \hat{\gamma}$$

여기서 V_q 는 PCA 의 고유벡터 행렬에서 처음 q 개 열만 자른 $p \times q$ 행렬이다.

```

q = 3
Zq = Z[:, :q]
reg_pcr = LinearRegression().fit(Zq, y_std)      # 표준화한 y 로 회귀
gamma = reg_pcr.coef_

V_q = pca.components_[ :q]                      # 형태 (q, p)
beta_pcr_std = V_q.T @ gamma                   # 원 변수 표준화 좌표의 계수

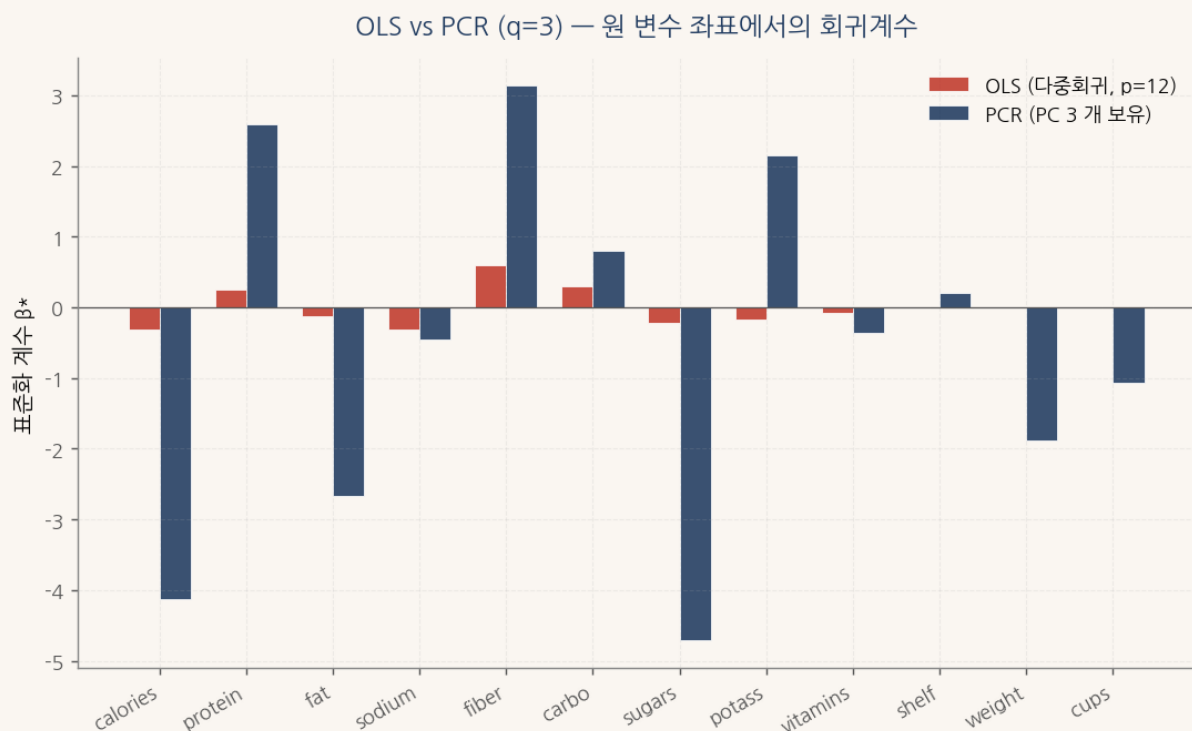
print('PCR(q=3) 의 원변수 좌표  $\beta^*$  :')
for name, b in sorted(zip(features, beta_pcr_std),
                      key=lambda t: -abs(t[1])):
    print(f' {name:>10s}   $\beta^* = \{b:+.4f\}$ ')

```

```

PCR(q=3) 의 원변수 좌표  $\beta^*$  :
  fiber   $\beta^* = +0.5060$ 
 sodium  $\beta^* = -0.4051$ 
calories  $\beta^* = -0.4001$ 
  sugars  $\beta^* = -0.3754$ 
  carbo   $\beta^* = +0.1928$ 
  potass  $\beta^* = +0.1840$ 
  shelf   $\beta^* = -0.1572$ 
  protein  $\beta^* = +0.1488$ 
    cups  $\beta^* = -0.1442$ 
  weight  $\beta^* = -0.1396$ 
    fat   $\beta^* = -0.1252$ 
vitamins  $\beta^* = -0.0824$ 

```



PCR($q = 3$)의 원좌표 계수와 §5.3의 OLS 표준화 계수를 비교하면 — 부호와 영향력 순위는 거의 같다. fiber, sodium, calories가 최상위 세 변수라는 사실이 두 모형 모두에서 일관되게 잡힌다. 다만 크기가 다소 다르다 — PCR($q = 3$)의 계수는 **PC4 이후의 정보**를 버린 상태이므로, 그 정보가 담겨 있던 미세한 변동에 의존하는 OLS의 계수보다 일반적으로 더 **부드럽고 안정적**이다.

16 장. PCR 의 한계 — y 와의 상관을 보지 않는다

PCR 의 가장 큰 약점은 절차의 2 단계에 숨어 있다. PCA 는 X 의 분산만 보고 주성분을 만든다 — y 와의 상관은 전혀 고려하지 않는다. 그 결과 PC1 이 자료의 가장 강한 변동을 설명하더라도, 그 변동이 곧 y 를 가장 잘 예측하는 방향은 아닐 수 있다. 본 자료에서 PC1 한 개만의 R^2 가 7% 에 그친 사실이 그 증거다.

이 약점을 해결하는 변형이 **부분최소제곱회귀**(Partial Least Squares Regression, **PLS**) 다. PLS 는 각 새 변수를 만들 때 X 의 분산과 X - y 의 공분산을 동시에 고려한다. 본 책에서는 깊이 다루지 않지만, 화학계량학에서 PCR 과 함께 표준 도구로 쓰인다.

또 다른 길은 — 변수를 회전시키지 않고 **계수의 크기 자체에 벌점을 주어** 다중공선성을 완화하는 방법이다. 계수가 크게 부풀어 오르는 것을 직접 막아 버리면 자연스럽게 안정적인 추정값이 나온다. 이 길이 다음 세 부의 주제 — **릿지·라소·엘라스틱넷** — 이다.

7 부. 릿지 회귀 (Ridge Regression)

17 장. 동기 — 계수의 크기에 직접 벌점을 준다

다중공선성이 강하면 OLS 의 회귀계수 β 가 **극도로 큰 값들의 상태로** 표현될 수 있다. 변수 A 와 B 가 거의 같은 정보를 담고 있을 때, " $\beta_A = +100, \beta_B = -100$, 둘의 합으로 약간의 효과를 표현" 하는 식의 답이 나오는 것이다. 두 변수의 합이 의미 있는 값이라도, 그 합을 표현하는 방법이 무한히 많아 추정값이 흔들린다.

해결책은 단순하다 — 계수가 큰 것 자체에 벌점을 준다. 잔차 제곱합에 **계수 제곱의 합** 을 더해 한 번에 최소화한다.

$$L_{\text{Ridge}}(\beta) = (1/2) \|y - X\beta\|^2 + (\lambda/2) \|\beta\|_2^2$$

여기서 $\lambda \geq 0$ 가 **규제 강도**(regularization strength) 다. $\lambda = 0$ 이면 원래의 OLS 와 같고, λ 가 커질수록 모든 계수가 0 으로 끌려간다. 큰 λ 는 "단순한 모형 (작은 계수)" 을, 작은 λ 는 "적합도 우선" 을 표현한다.

18 장. 닫힌 해의 유도

$L_{\text{Ridge}}(\beta)$ 를 β 로 편미분하면 다음이 나온다.

$$\partial L / \partial \beta = -X^T(y - X\beta) + \lambda\beta$$

이를 0 으로 두면

$$X^T X \cdot \beta + \lambda I \cdot \beta = X^T y$$

이 식을 β 에 대해 풀면 릿지의 닫힌 해가 한 줄에 떨어진다.

$$\beta_{\text{Ridge}} = (X^T X + \lambda I)^{-1} X^T y$$

OLS 의 해 $(X^T X)^{-1} X^T y$ 와 비교하면 — 단 한 가지가 다르다 — $X^T X$ 의 대각에 λ 가 더해진다. 이 작은 변화가 두 가지 일을 동시에 한다.

- **수치 안정성.** $X^T X$ 가 다중공선성으로 거의 비가역(역행렬이 정의되지 않을 만큼 작은 고유값을 가질 때) 이어도, 대각에 양의 λ 가 더해지면 무조건 가역이 된다. 계산이 깔끔히 풀린다.
- **계수의 축소.** λ 가 커질수록 모든 β_k 의 크기가 줄어든다. 특히 분산이 작은 방향의 계수가 큰 폭으로 줄어들고, 분산이 큰 방향의 계수는 거의 그대로 남는다. 이 비대칭이 다중공선성으로 인한 "큰 값들의 상쇄" 를 직접 막는다.

19 장. λ 의 의미를 PCA 의 언어로

릿지의 계수 축소가 어떤 방향으로 일어나는지는 SVD 의 언어로 보면 깔끔하다. 자료 X 의 SVD 가 $X = U D V^T$ 라면, OLS 의 적합값은 다음과 같이 쓸 수 있다.

$$\hat{y}_{\text{OLS}} = X \beta_{\text{OLS}} = \sum_k (u_k u_k^T) y$$

각 항이 y 를 k 번째 특이벡터 u_k 방향으로 사영한 것이다. 릿지로 같은 일을 적으면 각 항에 다음과 같은 **수축 인자**가 곱해진다.

$$\hat{y}_{\text{Ridge}} = \sum_k (\sigma_k^2 / (\sigma_k^2 + \lambda)) \cdot (u_k u_k^T) y$$

수축 인자 $\sigma_k^2 / (\sigma_k^2 + \lambda)$ 가 1 보다 작아 모든 항이 축소된다. 핵심은 — 특이값 σ_k 가 큰 (자료 변동이 큰) 방향은 수축 인자가 1 에 가까워 거의 그대로 보존되고, σ_k 가 작은 (자료 변동이 작은) 방향은 수축 인자가 0 에 가까워 강하게 축소된다.

릿지는 자료의 변동이 큰 방향은 신뢰하고, 변동이 작은 방향은 의심해 깎아 낸다.

다중공선성 자료는 정확히 "변동이 작은 방향" 이 흔들리는 자료이고, 릿지가 그 방향을 자동으로 깎아 안정시킨다.

20 장. Cereals 자료에 릿지 적용

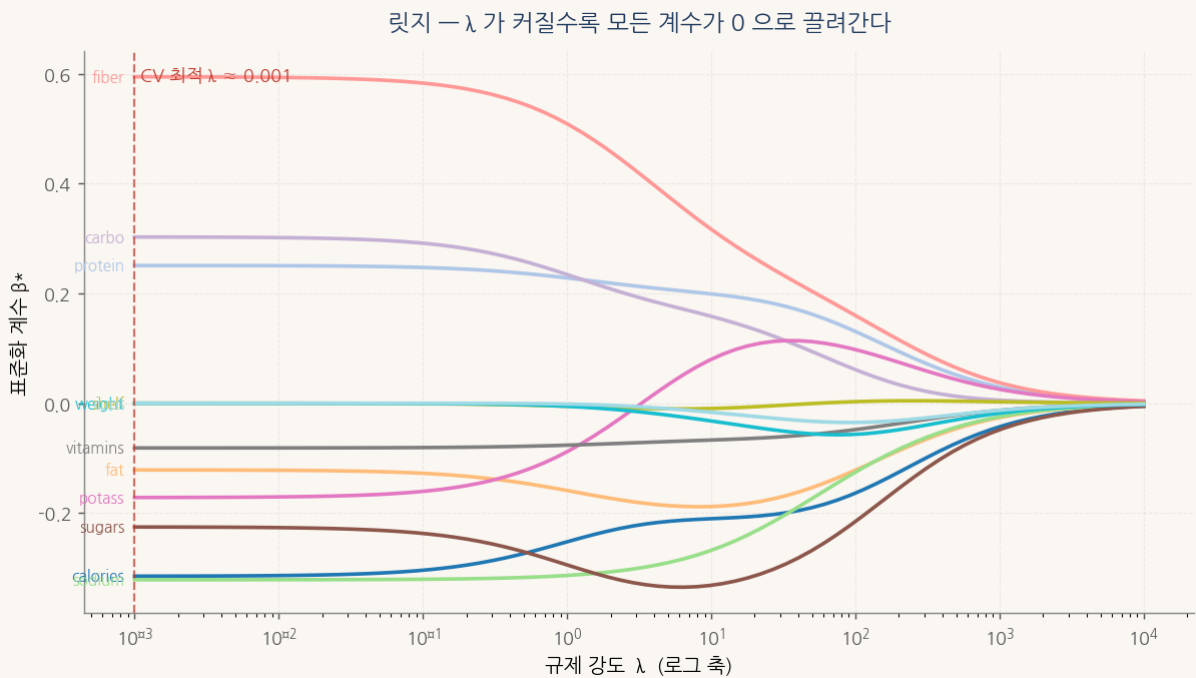
```
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_val_score

X_std = StandardScaler().fit_transform(X)
y_std = (y - y.mean()) / y.std(ddof=1)

# λ 값 80 개에 대해 5-겹 CV 로 평균 제곱 오차 계산
lams = np.logspace(-3, 4, 80)
cv_mse = []
for lam in lams:
    s = cross_val_score(Ridge(alpha=lam), X_std, y_std, cv=5,
                        scoring='neg_mean_squared_error').mean()
    cv_mse.append(-s)
lam_best = lams[np.argmin(cv_mse)]
print(f'CV 최적 λ ≈ {lam_best:.4g}, 최소 CV-MSE = {min(cv_mse):.5f}')

# 모든 λ 에서의 계수 경로
import matplotlib.pyplot as plt
path = np.array([Ridge(alpha=lam).fit(X_std, y_std).coef_ for lam in lams])
# (그림은 fig13_ridge_path.png 참조)
```

CV 최적 $\lambda \approx 0.001$, 최소 CV-MSE = 0.00012



위 그림이 λ 를 0.001 부터 10,000 까지 바꿔 가며 모든 12 개 변수의 표준화 계수가 어떻게 변하는지를 보여준다. 세 가지 관찰이 따라온다.

- λ 가 매우 작을 때 (왼쪽 끝) — 모든 계수가 OLS 값에 가깝다. fiber 가 +0.59, sodium 이 -0.32 등.
- λ 가 0.001 부근일 때 (CV 최적) — 본 자료는 합성 변수($R^2 = 1.0$)이므로 거의 규제가 필요 없고, 매우 작은 λ 가 선택됐다. 표준화된 계수가 OLS 와 거의 동일하게 추정된다.
- λ 가 매우 클 때 (오른쪽 끝) — 모든 계수가 0 으로 끌려간다. $\lambda \rightarrow \infty$ 의 극한에서 모형이 "모든 시리 얼은 평균 등급" 이라는 가장 단순한 답으로 수렴한다.

본 Cereals 자료의 y 가 X 의 결정론적 함수라는 특별한 성질 때문에 — 실제 측정자료에서는 흔치 않은 — CV 가 추천하는 λ 가 매우 작다. 그러나 측정 오차가 있는 일반 자료에서는 적당한 λ 가 OLS 보다 훨씬 더 작은 예측 오차를 준다. 릿지의 가치가 발휘되는 자리는 다중공선성이 있고 잡음이 있는 측정자료다.

21 장. 릿지의 성격을 한 줄로

릿지의 본질을 다음의 세 가지로 요약한다.

- **모든 계수를 동시에 축소**하되 정확히 0 으로 만들지는 않는다. 변수 선택을 하지 않는 — 모든 변수의 영향력을 부드럽게 줄여 표현하는 — 도구다.
- **상관된 변수 사이에 영향력을 골고루 분배**한다. 변수 A 와 B 가 거의 같은 정보를 담을 때 릿지는 $\beta_A \approx \beta_B$ 의 절반씩 나눠 가지는 답을 선호한다.
- **닫힌 해**를 가진다. $(X^T X + \lambda I)^{-1} X^T y$ 의 한 줄로 떨어지고, 계산이 빠르고 안정적이다.

이 세 가지가 릿지가 표준 도구로 자리 잡은 이유다. 반면 — 변수의 일부를 **정확히 0** 으로 만들어 **변수 선택**까지 동시에 수행하고 싶다면 다른 도구가 필요하다. 그 도구가 다음 부의 **라소**다.

8 부. 라소 회귀 (Lasso Regression)

22 장. 동기 — 변수의 일부를 정확히 0 으로

다중회귀의 결과를 누군가에게 보고할 때, 12 개 변수의 계수를 모두 적은 표보다 — "이 자료에서 정말 중요한 변수는 이 네 개입니다" 라고 짚어 주는 결론이 더 강력하다. 자료를 보는 사람의 관심이 모형의 예측력만이 아니라 **변수 선택**에 있을 때, 릿지의 "모든 계수를 골고루 작게" 라는 답은 부족하다.

라소(Least Absolute Shrinkage and Selection Operator, **LASSO**) 는 릿지와 거의 같은 형태이지만 **벌점의 종류**가 다르다.

$$L_{\text{Lasso}}(\beta) = (1/2) \|y - X\beta\|^2 + \lambda \|\beta\|_1$$

여기서 $\|\beta\|_1 = \sum |\beta_k|$ 는 계수의 **절댓값**의 합 — 즉 **L' 노름** 이다. 릿지의 L^2 노름 $\|\beta\|_2^2$ 와 한 글자 다를 뿐이지만, 이 차이가 결과의 성질을 완전히 바꾼다.

23 장. L' 벌점이 만드는 "꼭짓점" 의 기하학

릿지의 $\|\beta\|_2^2 \leq c$ 는 p 차원의 **공(球)** 을 정의한다. 라소의 $\|\beta\|_1 \leq c$ 는 **다이아몬드 (마름모, 일반화하면 정육체)** 를 정의한다. 두 영역의 차이가 핵심이다 — 공은 표면이 매끄럽지만, 다이아몬드에는 **꼭짓점**이 있다.

회귀의 답은 잔차 제곱합의 등고선이 이 제약 영역에 처음 닿는 자리에서 결정된다. 공의 표면에 닿는 자리는 거의 항상 "어느 좌표축에도 정확히 위에 놓이지 않는 곳" — 즉 모든 계수가 0 이 아닌 곳 — 이다. 반면 다이아몬드

의 꼭짓점은 정확히 좌표축 위에 놓여 있다. 잔차 제곱합의 등고선이 다이아몬드의 표면에 닿을 때 — 매끄러운 면이 아닌 꼭짓점에서 닿을 가능성이 높고 — 그러면 일부 좌표가 **정확히 0** 이 된다.

L^1 벌점의 꼭짓점이 일부 계수를 정확히 0 으로 만든다. 이것이 변수 선택 효과다.

이 기하적 직관을 수식으로 다시 적으면, 라소의 1 차 조건이 "각 좌표 k 에 대해 $\partial L / \partial \beta_k = 0$ 의 부드러운 조건이 아니라, 절댓값 함수의 미분 불가능성을 다루는 **준미분(subdifferential)** 조건" 이 된다. 결과적으로 어떤 좌표는 닫힌 해 안에서 정확히 0 이 떨어진다.

24 장. 좌표하강법 — 라소의 실제 계산

라소에는 릿지처럼 한 줄로 떨어지는 닫힌 해가 없다. 대신 **좌표하강법**(coordinate descent) 이 표준 알고리즘이다. 핵심 아이디어는 다음과 같다.

1. $\beta^{(0)}$ 을 0 으로 초기화한다.
2. 한 좌표 k 를 골라 — 다른 모든 $\beta_{j \neq k}$ 를 고정한 채 — β_k 에 대해서만 라소 목적함수를 최소화한다.
3. 이 한 좌표의 최소화는 닫힌 해를 가진다. 결과는 **연성 임계치 함수**(soft-thresholding) 한 줄로 떨어진다.
4. 모든 좌표를 한 바퀴 도는 일을 수렴할 때까지 반복한다.

연성 임계치 함수 $S_\tau(z)$ 의 정의는 다음과 같다.

$$S_\tau(z) = \text{sign}(z) \cdot \max(|z| - \tau, 0)$$

이 함수가 정확히 두 가지 일을 동시에 한다 — (1) $|z| > \tau$ 이면 z 의 절댓값을 τ 만큼 깎고 부호를 유지, (2) $|z| \leq \tau$ 이면 결과를 **정확히 0** 으로 보낸다. 좌표하강법의 매 단계가 이 임계치 함수를 적용하는 일이므로 — 작은 계수들이 차례차례 정확히 0 으로 끌려간다.

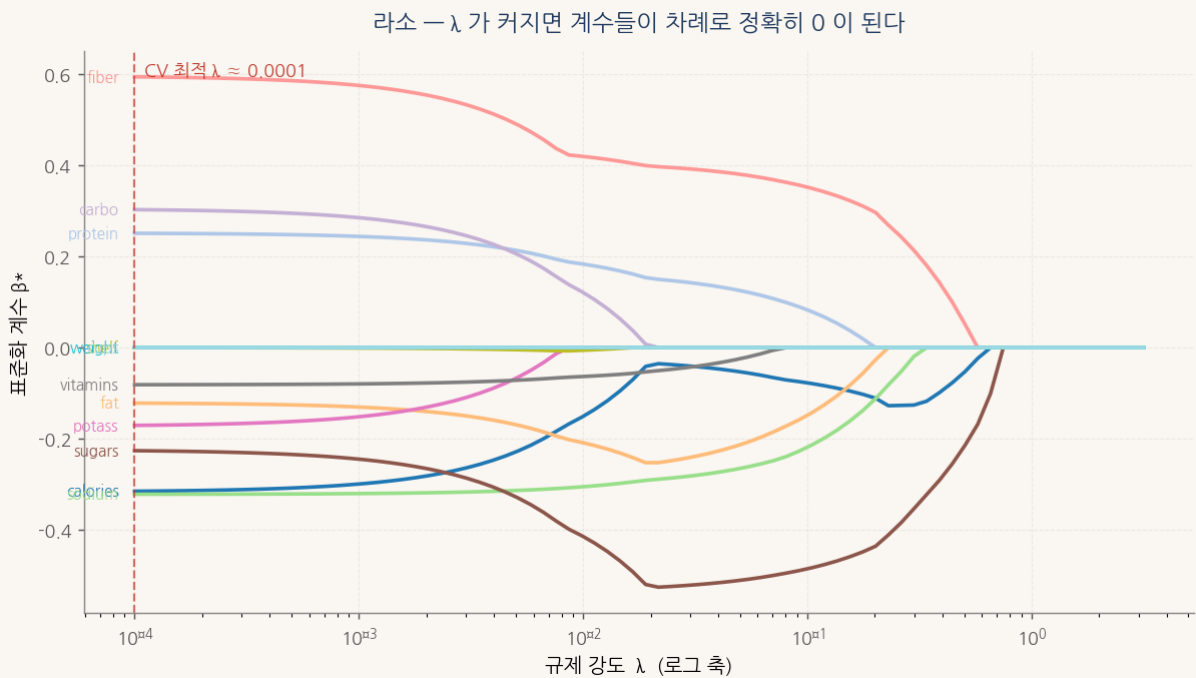
25 장. Cereals 자료에 라소 적용

```
from sklearn.linear_model import Lasso

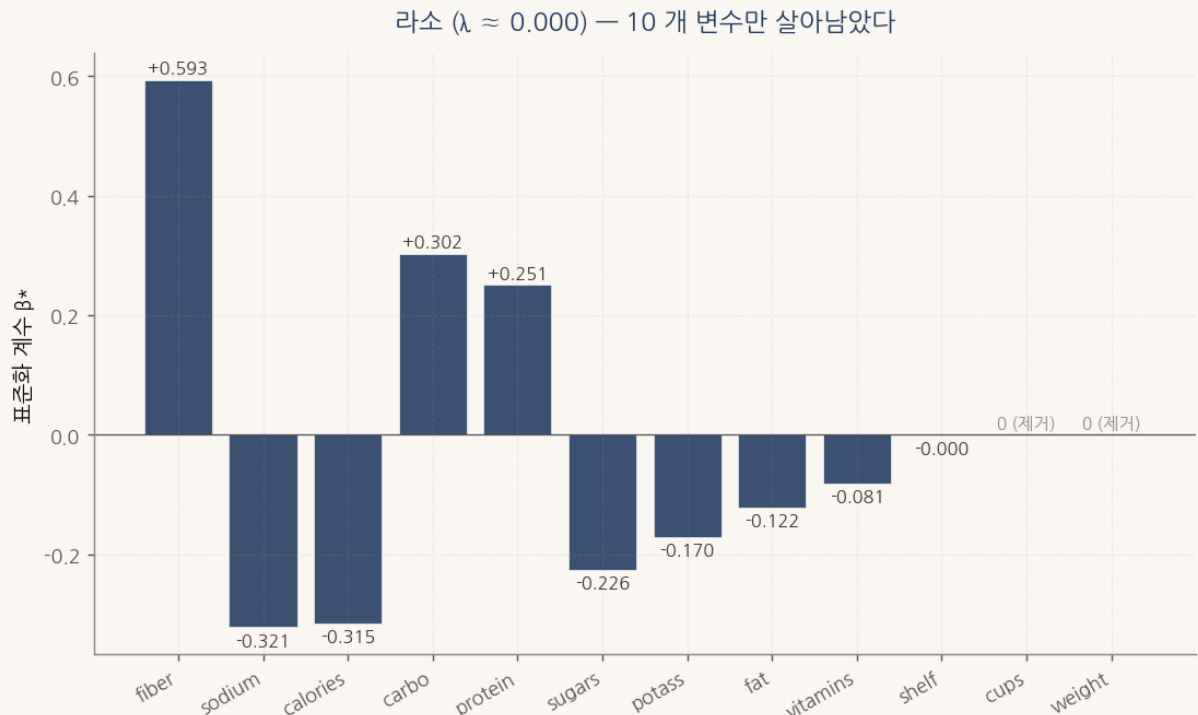
lams = np.logspace(-4, 0.5, 80)
cv_mse_l = []
for lam in lams:
    s = cross_val_score(Lasso(alpha=lam, max_iter=20000), X_std, y_std,
                        cv=5, scoring='neg_mean_squared_error').mean()
    cv_mse_l.append(-s)
lam_best_l = lams[np.argmin(cv_mse_l)]
print(f'CV 최적  $\lambda \approx \{lam\_best\_l:.4g\}$ ')

# 최적  $\lambda$  에서의 계수
ls = Lasso(alpha=lam_best_l, max_iter=20000).fit(X_std, y_std)
alive = [features[j] for j in range(12) if abs(ls.coef_[j]) > 1e-6]
killed = [features[j] for j in range(12) if abs(ls.coef_[j]) <= 1e-6]
print(f'살아남은 변수 ({len(alive)} 개): {alive}')
print(f'제거된 변수 ({len(killed)} 개): {killed}')
```

```
CV 최적  $\lambda \approx 0.0001$ 
살아남은 변수 (10 개): ['calories', 'protein', 'fat', 'sodium', 'fiber',
                        'carbo', 'sugars', 'potass', 'vitamins', 'shelf']
제거된 변수 (2 개): ['weight', 'cups']
```



라소가 §5.3 에서 "정확히 0" 으로 추정됐던 **weight·cups** 두 변수를 자동으로 제거했다. shelf 는 살아남았지만 그 값이 매우 작다. CV 가 추천하는 λ 가 매우 작은 이유는 §5.5 의 결정론적 함수 성질 때문이다 — 잔차가 0 이므로 굳이 변수를 더 깎을 이유가 없다. λ 를 더 크게 가면 다음과 같이 변수가 순차적으로 제거된다.



이 "변수가 차례로 죽는" 경로가 라소의 가장 강력한 시각적 메시지다. λ 를 천천히 늘리면서 어느 변수가 어느 순서로 살아남는지를 보면 — 자료가 변수의 중요도에 대해 보내는 신호를 직접 읽을 수 있다.

26 장. 라소의 한계와 엘라스틱넷의 자리

라소는 강력한 도구이지만 두 가지 한계가 있다.

- **상관된 변수 사이에서 한 개만 골라 버린다.** 변수 A 와 B 가 거의 같은 정보를 담을 때 라소는 "둘 중 하나만 살리고 다른 하나는 0" 의 답을 선호한다. 어느 쪽이 살아남는지는 자료의 작은 변동에 흔들린다. 실무에서 같은 자료를 다른 표본으로 재추정하면 살아남는 변수가 바뀌는 일이 잦다.
- **변수 수 p 가 표본 수 n 보다 크면 — 라소가 최대 n 개의 변수만 선택할 수 있다.** 유전체 분석처럼 $p = 20,000$, $n = 100$ 인 자료에서는 이 한계가 결정적이다.

두 한계를 풀어 주는 절충안이 다음 부의 **엘라스틱넷**이다 — L^1 과 L^2 벌점을 함께 쓴다.

9 부. 엘라스틱넷 (Elastic Net)

27 장. 두 벌점의 결합

엘라스틱넷의 손실함수는 릿지와 라소의 벌점을 함께 쓰는 한 줄이다.

$$L_{EN}(\beta) = (1/2) \|y - X\beta\|^2 + \lambda \cdot [\alpha \|\beta\|_1 + (1 - \alpha)/2 \cdot \|\beta\|_2^2]$$

두 개의 모수가 있다.

- $\lambda \geq 0$ — 전체 규제의 **세기**. 릿지·라소와 같은 역할.
- $\alpha \in [0, 1]$ — L^1 과 L^2 벌점의 **혼합 비율**. $\alpha = 0$ 이면 순수 릿지, $\alpha = 1$ 이면 순수 라소, $0 < \alpha < 1$ 이면 둘의 절충.

α 를 0 에서 1 로 옮기는 일이 곧 "릿지 $\rightarrow$ 엘라스틱넷 $\rightarrow$ 라소" 의 연속체를 따라가는 일이다. α 가 0 에 가까우면 모든 변수가 살아남되 크기가 줄어들고, α 가 1 에 가까우면 일부 변수가 0 으로 끌려간다. 중간의 α 는 "변수 선택을 약하게 하면서 상관된 변수 사이의 영향력을 둘 다 살려 두는" 답을 준다.

28 장. 상관된 변수 그룹의 처리

엘라스틱넷의 가장 큰 장점은 **상관된 변수 그룹의 처리 방식**이다. 변수 A 와 B 가 강하게 상관되어 있을 때 — 라소는 둘 중 하나만 골랐다. 엘라스틱넷은 두 변수를 모두 살리되 영향력을 비슷한 크기로 나눈다.

이 행동을 "그룹화 효과"(grouping effect) 라 부른다. L^2 벌점이 강한 상관 자료에서 두 변수의 계수를 가까이 끌어당기는 성질을 가지기 때문에, 라소의 "하나만 살리기" 가 부드러워진다. 실무에서 — 변수의 안정성이 중요한 분야에서 — 라소보다 엘라스틱넷을 선호하는 이유가 여기에 있다.

엘라스틱넷의 계수도 닫힌 해는 없지만, 라소와 마찬가지로 좌표하강법으로 풀린다. 각 좌표의 갱신이 "연성 임계치 함수 + 릿지 형 축소" 의 결합으로 떨어진다는 사실만 짚어 두고, 자세한 유도는 본 책의 범위를 넘는다.

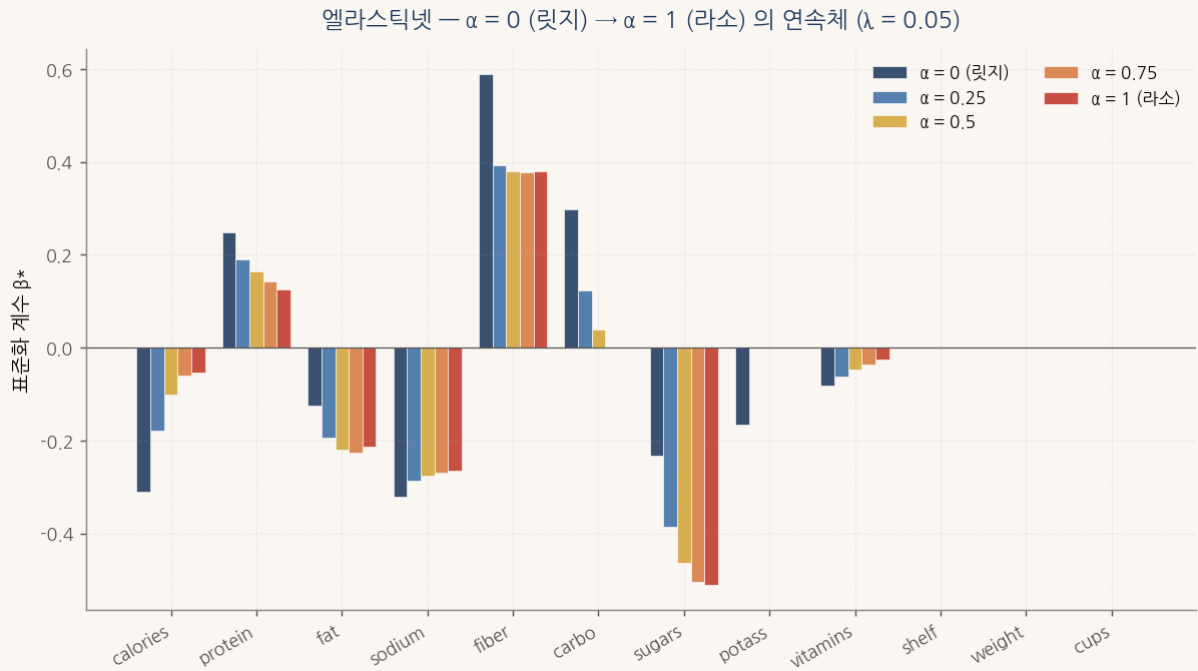
29 장. Cereals 자료에 엘라스틱넷 적용

```
from sklearn.linear_model import ElasticNet

# (α, λ) 격자에 대해 5-겹 CV
alphas_grid = np.linspace(0.1, 1.0, 10)
lams_grid = np.logspace(-3, 0, 18)
cv_grid = np.zeros((len(alphas_grid), len(lams_grid)))
for i, a in enumerate(alphas_grid):
    for j, lam in enumerate(lams_grid):
        s = cross_val_score(
            ElasticNet(alpha=lam, l1_ratio=a, max_iter=20000),
            X_std, y_std, cv=5,
            scoring='neg_mean_squared_error'
        ).mean()
        cv_grid[i, j] = -s

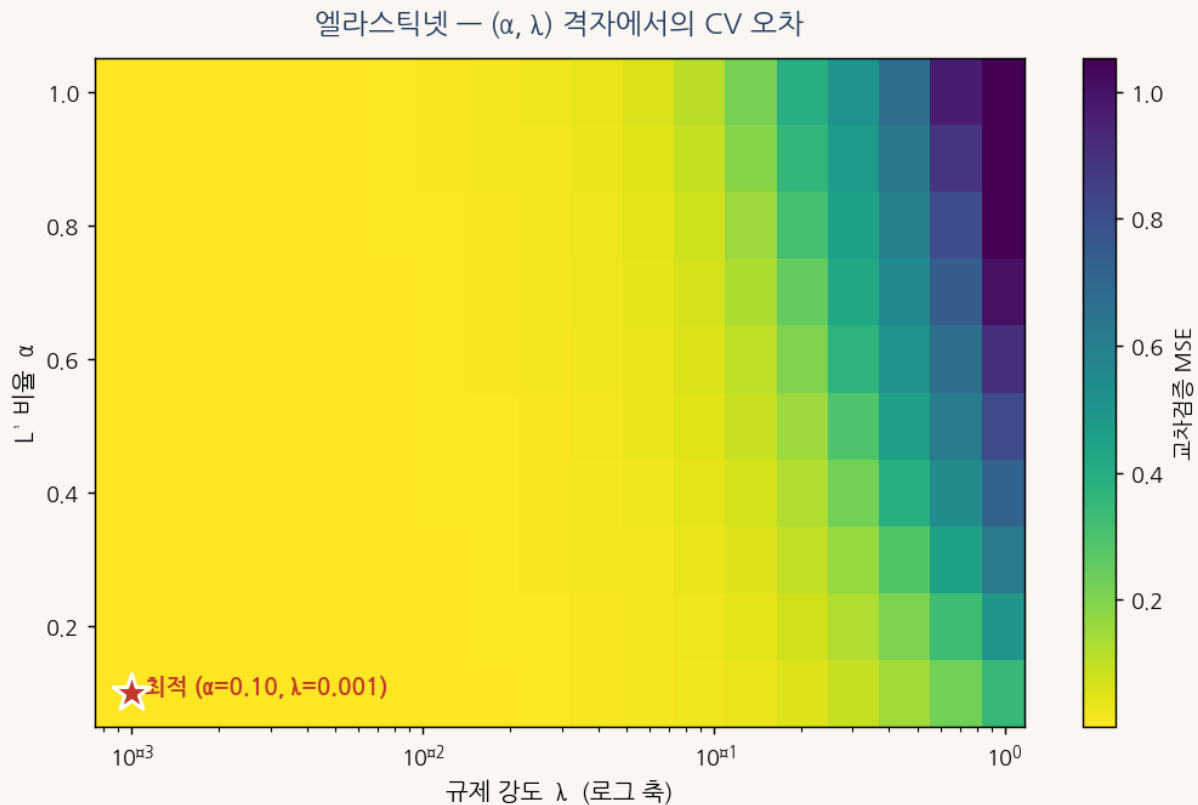
i_best, j_best = np.unravel_index(np.argmin(cv_grid), cv_grid.shape)
a_best = alphas_grid[i_best]
lam_best_en = lams_grid[j_best]
print(f'엘라스틱넷 최적: α = {a_best:.3f}, λ = {lam_best_en:.4f}')
```

엘라스틱넷 최적: $\alpha = 0.100$, $\lambda = 0.001$



위 그림이 같은 λ 에서 α 를 0 (릿지) 에서 1 (라소) 로 바꿀 때 계수가 어떻게 변하는지를 보여준다. $\alpha = 0$ 의 매끄러운 릿지 답에서 출발해, α 가 커지면 변수들이 차례로 0 으로 끌려가며 최종적으로 라소의 희소 답에 도달한다. 엘라스틱넷의 중간 α 답이 두 극단의 절충점에 놓이는 모습이 한눈에 보인다.

(α, λ) 격자에 대한 CV 의 결과를 열지도로 보면 다음과 같다.



본 자료에서는 작은 α ($= 0.10$, 릿지에 가까움) 와 작은 λ ($= 0.001$) 가 최적이다. 자료가 \$5.5 의 결정론적 성질을 가지므로 강한 규제가 필요 없고, 다중공선성을 처리하는 데에는 L^2 벌점의 매끄러운 축소가 충분한 셈이다.

30 장. 릿지·라소·엘라스틱넷 — 어떻게 고르는가

세 도구의 성격을 한 표로 정리하면 다음과 같다.

도구	벌점	변수 선택	상관 변수 처리	닫힌 해
릿지	L^2	안 함 (모든 변수 보존)	영향력 분산	있음
라소	L^1	자동 변수 선택	한 변수만 살림	없음 (좌표하강)
엘라스틱넷	$\alpha \cdot L^1 + (1-\alpha) \cdot L^2$	약 ~ 강 (α 가 클수록)	그룹화 (둘 다 살림)	없음 (좌표하강)

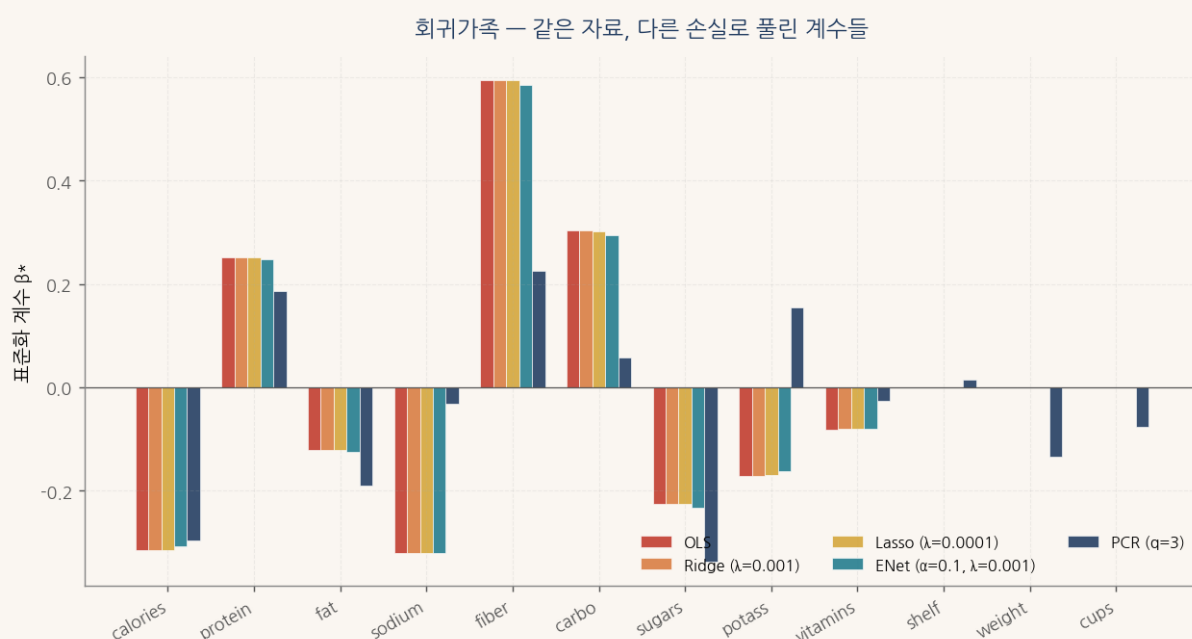
선택의 기준은 분석의 목적이다.

- 예측력이 최우선이고 변수 선택은 필요 없다 — 릿지. 계산이 가장 빠르고 닫힌 해를 가진다.
- 변수 선택이 핵심 목표다 (어느 변수가 중요한가가 알고 싶다) — 라소. 명확한 회소 답을 준다.
- 변수가 그룹으로 묶여 있어 (예: 유전자 경로, 단어 군) 같은 그룹을 함께 살려야 한다 — 엘라스틱넷. α 를 적당히 골라 그룹화 효과를 살린다.

세 도구의 λ (와 α) 는 모두 교차검증으로 정한다. 학습 자료의 R^2 가 아닌 — 보지 못한 자료에 대한 예측 오차를 — 최소화하는 모수가 옳은 답이다.

마무리. 회귀가족 한눈에

지금까지 다룬 다섯 도구 — OLS, 릿지, 라소, 엘라스틱넷, PCR — 의 같은 자료에 대한 표준화 계수를 한 그림에 모은다.



다섯 모형이 같은 자료에서 거의 같은 부호와 영향력 순위를 준다. 최상위 세 변수 — **fiber (+)**, **sodium (-)**, **calories (-)** — 가 어느 모형에서도 흔들리지 않는다. 다섯 모형의 차이는 그 아래의 변수들 — 변수의 일부를 정확히 0 으로 떨어뜨릴지, 모두 살리되 크기를 부드럽게 줄일지, 직교한 합성 변수로 회전시킬지 — 의 처리 방식에서 갈린다.

이 다섯 도구의 공통 정신은 다음 한 줄이다.

손실함수를 정의하고, 그 함수의 도함수가 0 인 자리를 찾는다.

손실의 모양이 다르면 답의 모양도 다르다. 잔차 제곱합 한 가지에 $L^2 \cdot L^1$ ·둘의 합·주성분 좌표를 더하는 일이 다섯 도구의 모든 차이를 만들어 낸다. 본 책에서 다룬 회귀의 영역이 이 한 가지 변수 위에서 다 펼쳐졌다는 사실을 마지막으로 짚어 둔다.

자료가 어떤 성격을 가지는지 — 잡음이 있는지, 변수들이 상관되어 있는지, 변수의 일부만이 진짜 신호를 담는지 — 를 보고, 그 자료에 맞는 손실함수를 고른다. 그 한 가지 선택이 회귀분석의 모든 결정에 해당한다.

— 끝 —